

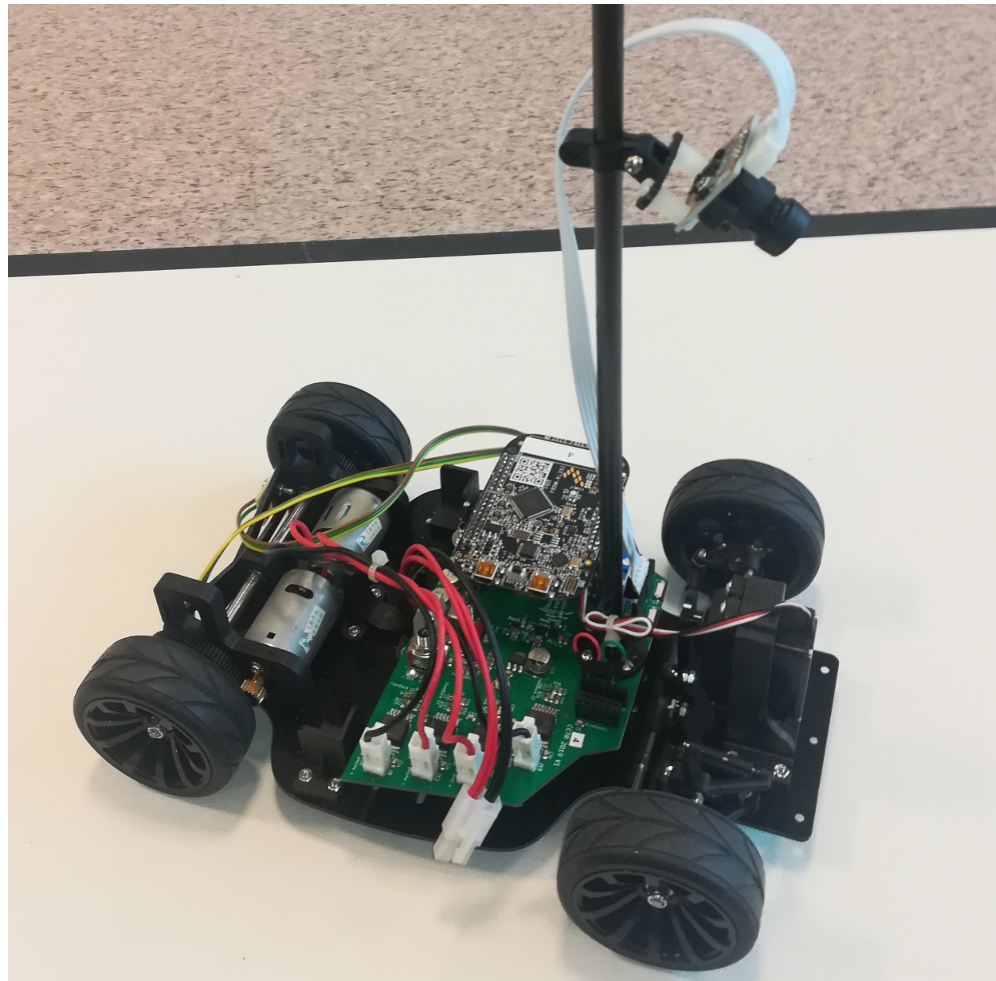
Contrôle-commande temps réel des systèmes Cyber-Physiques

S. Mancini



Résumé du module

Mise en pratique du contrôle-commande temps réel des systèmes Cyber-Physiques, sur une voiture autonome, avec un micro-contrôleur sous OS temps-réel.



Objectifs du projet

- ❑ Mise en oeuvre d'un système complet :
 - ★ Système physique: la voiture
 - ★ Microprocesseur
 - ★ Logiciel temps-réel et interface logiciel/matériel
- ❑ Programmation temps réel du contrôleur, sous FreeRTOS

Moyens pour le projet

- ❑ Voiture NXP modèle Alamak
- ❑ Microcontrôleur NXP FRDM-KL25Z
 - Environnement MCU Espresso (NXP)
 - OS Temps Réel FreeRTOS
- ❑ Carte d'interface spécifique
- ❑ Télémétrie par Bluetooth



Le matériel est mis à disposition par NXP en partenariat avec Phelma. Le département Formation Continue de G-INP a financé la nouvelle version.



Missions

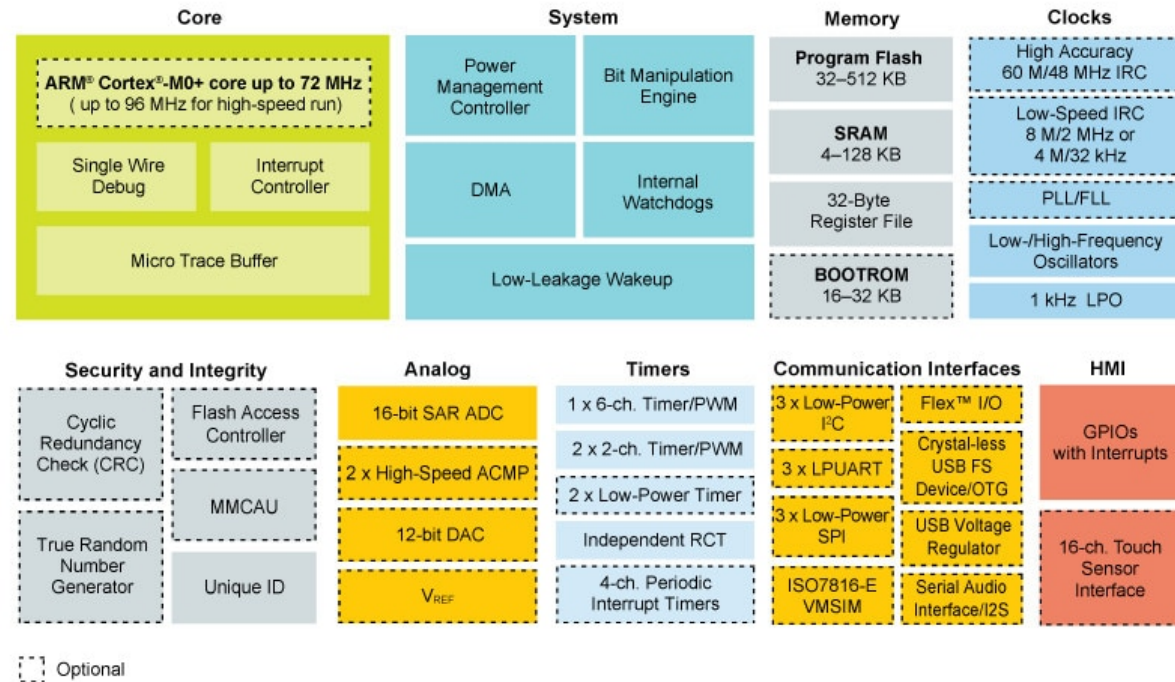
Concevoir des boucles de contrôle de plus en plus complexes pour pouvoir suivre la piste le plus vite possible:

❑ Niveau A :

- Prise en main du micro-contrôleur
- Rouler droit:
 - . Faire 1m en ligne droite
 - . et retour en arrière
 - . idem sur 4m

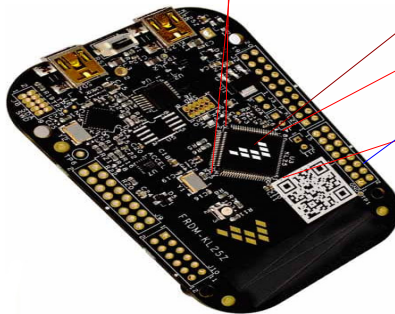
❑ Niveau B : ...

Micro-contrôleur

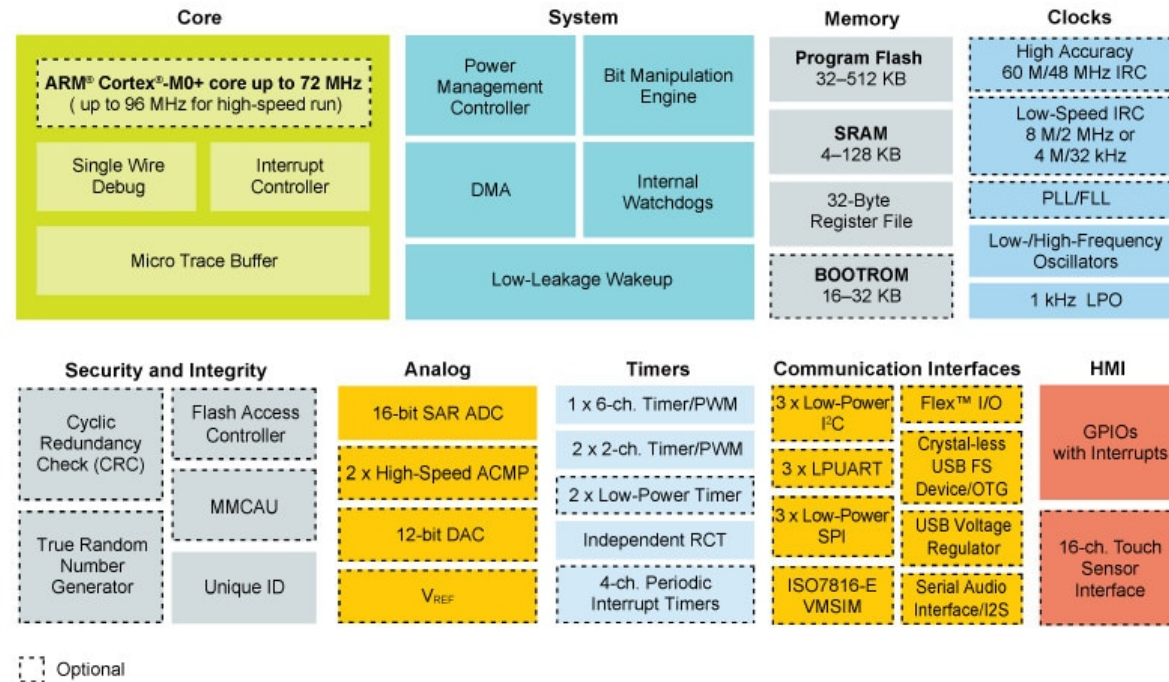


48 Mhz

I/O configurables



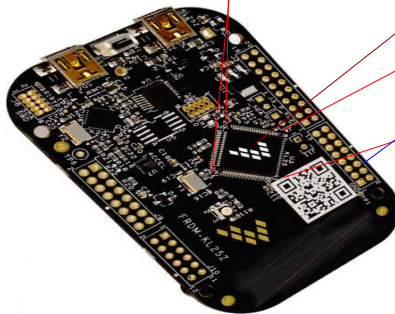
Micro-contrôleur



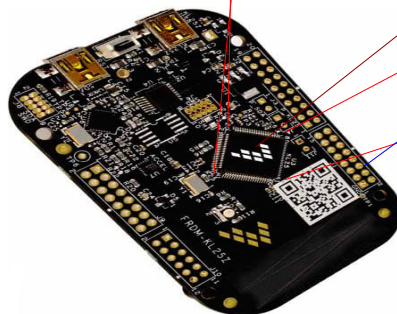
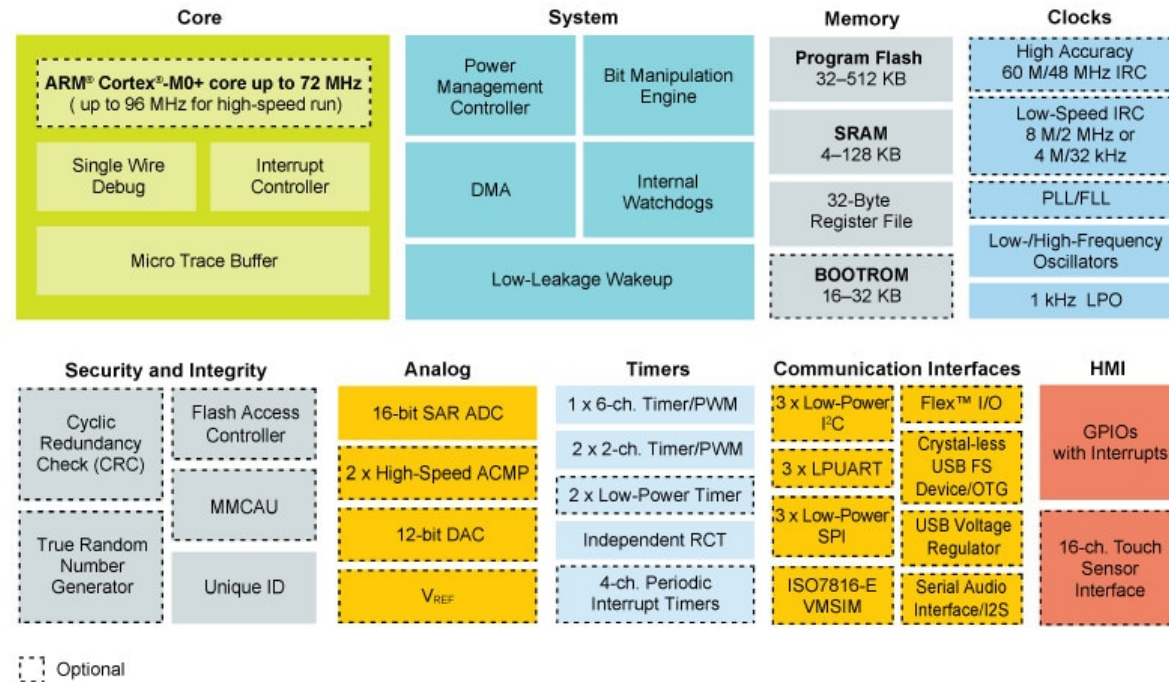
48 Mhz

I/O configurables

Pas de RAM externe



Micro-contrôleur

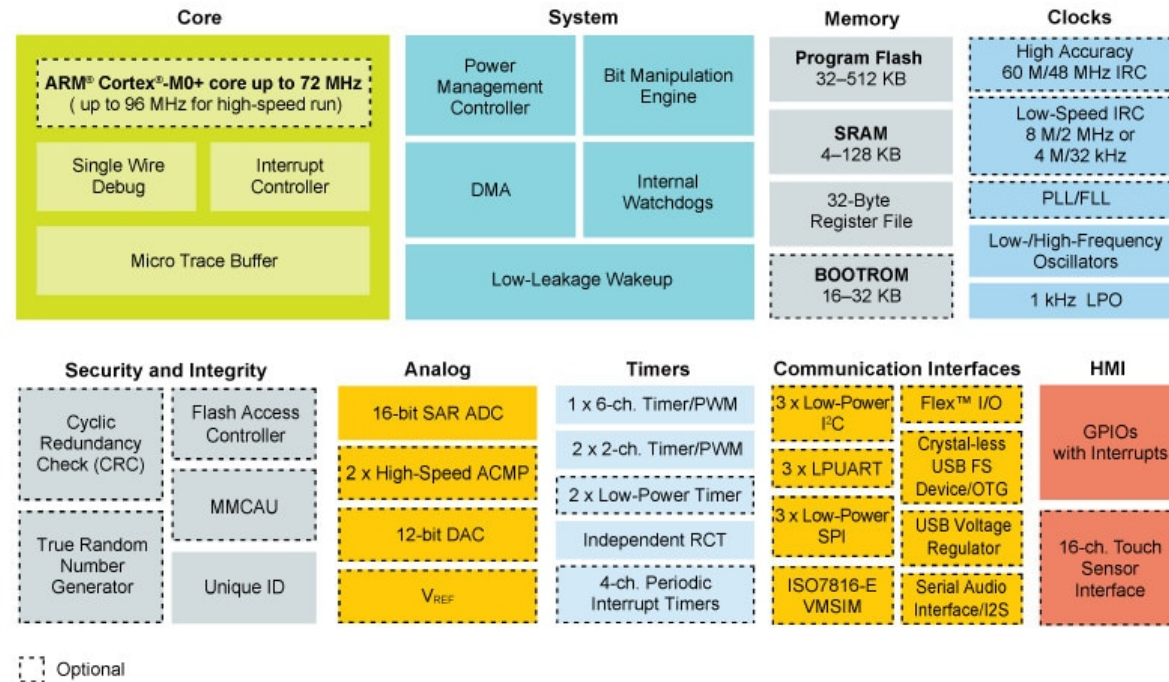


48 Mhz

I/O configurables

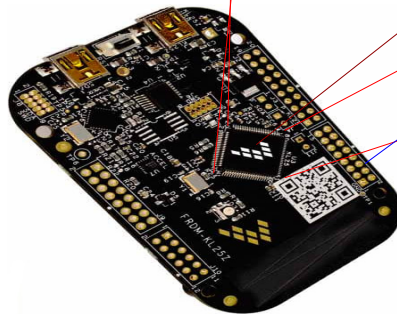
Pas de RAM externe
Pas de Disque Dur

Micro-contrôleur



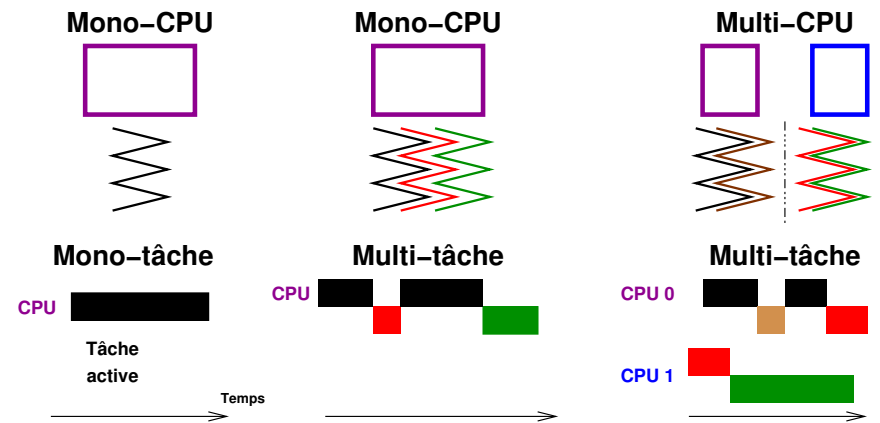
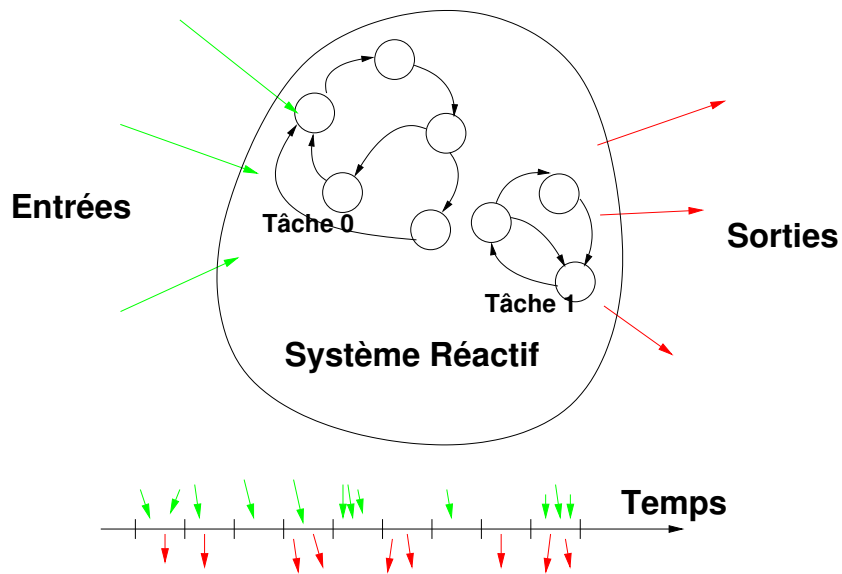
48 Mhz

I/O configurables



Pas de RAM externe
Pas de Disque Dur
Pas de MMU

Logiciel multi-tâche!



✖ Prise en main

- ✓ Rappels sur FreeRTOS
- C'est parti !!

□ Moteurs

□ Asservissement des moteurs

□ Contrôle de la direction

□ Planificateur de trajectoire

□ Gestion de la caméra

□ Suivi de la piste

Application sous FreeRTOS

```
int main(void) {
    /*
     * Fonctions d'initialisation des différentes entrées/sorties
     * et des protocoles de communication
     */
    BOARD_init_all();
    PRINTF("Hello World CCTR SEOC Test\n\r");
    /* Création d'une tâche avec pile statique, priorité statique */
    xTaskCreate(task_LED, "Task_LED",
               configMINIMAL_STACK_SIZE + 100, NULL,
               task_LED_PRIORITY, NULL);

    vTaskStartScheduler();
    /* Enter an infinite loop, but should never arrive here */
    for (;;)
        ;
    return 0 ;
}

int led_cpt=0;

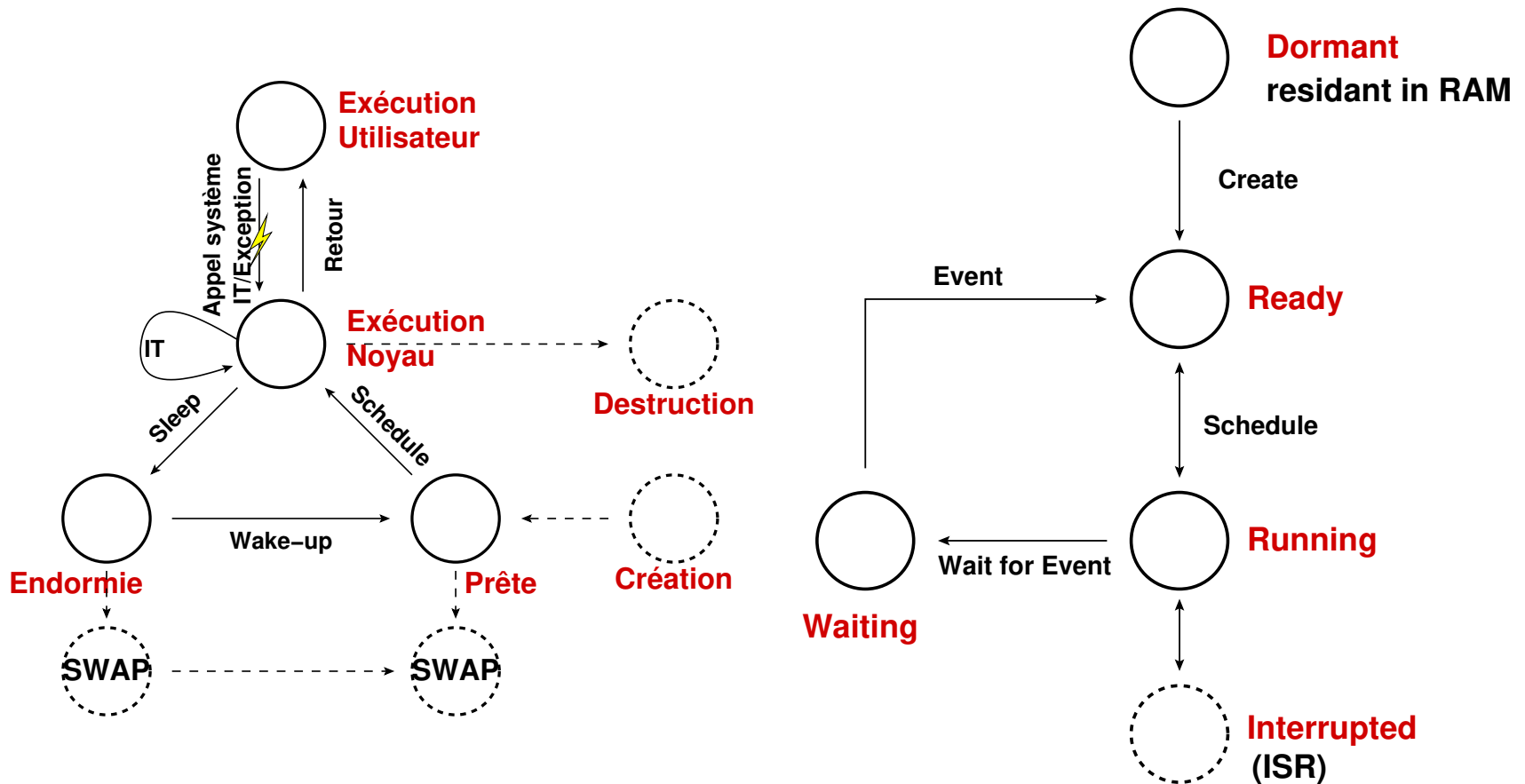
/*
 * Tâche périodique pour faire clignoter la LED
 */

void task_LED(void *pvParameters) {
    const TickType_t xDelay= Ms(100);
    led_cpt=0;

    for (;;)
    {
        LED_RED_TOGGLE();
        LED_GREEN_TOGGLE();
        LED_BLUE_TOGGLE();

        PRINTF("%d\r\n", led_cpt);
        vTaskDelay(xDelay);
        led_cpt++;
    }
}
```

Gestion des processus

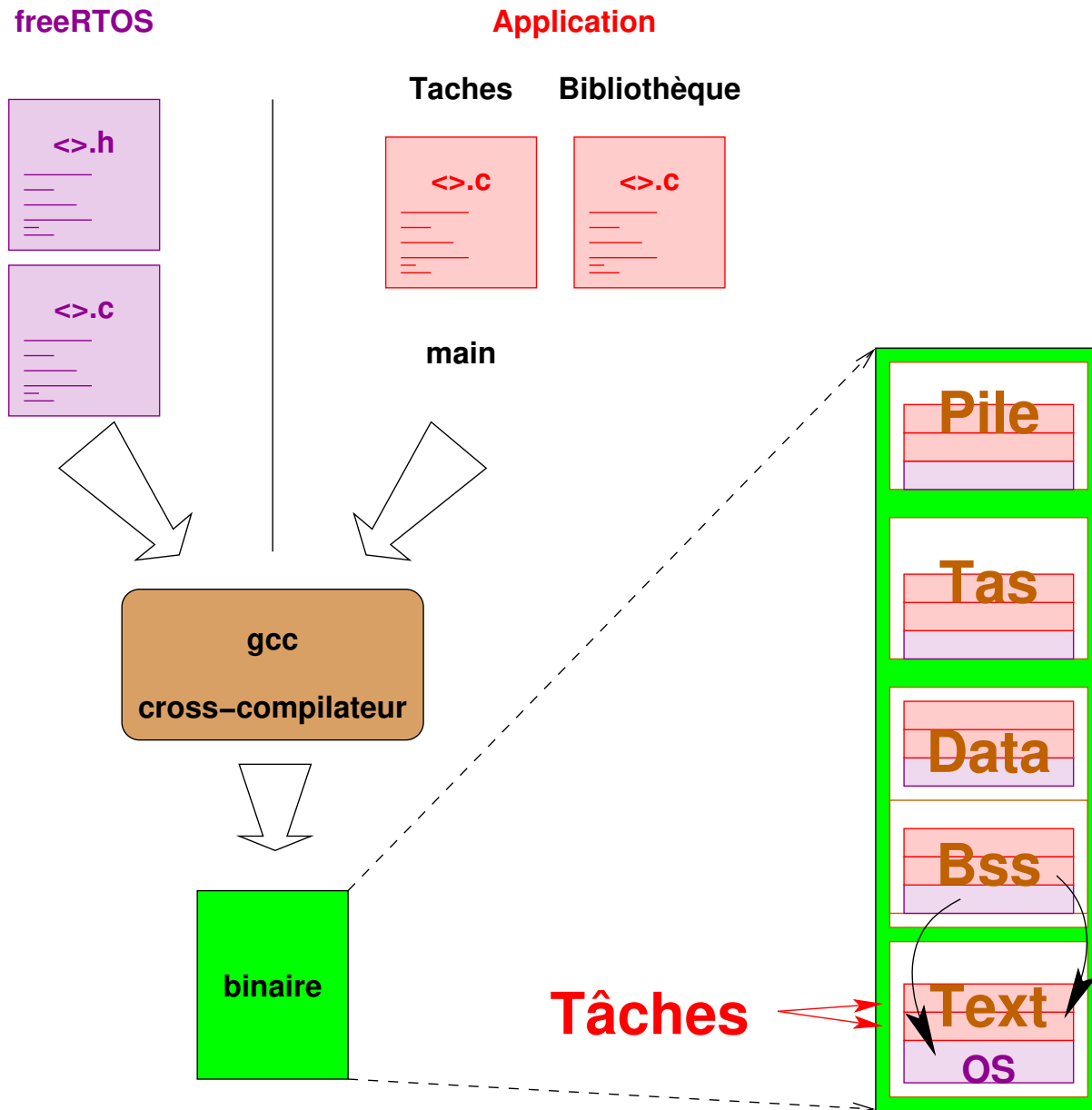


Etats des processus sous UNIX

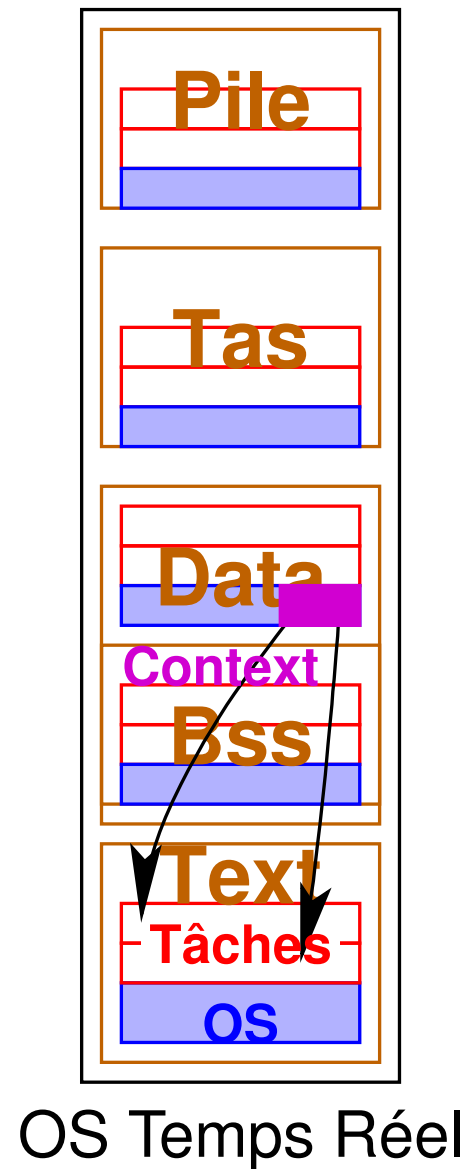
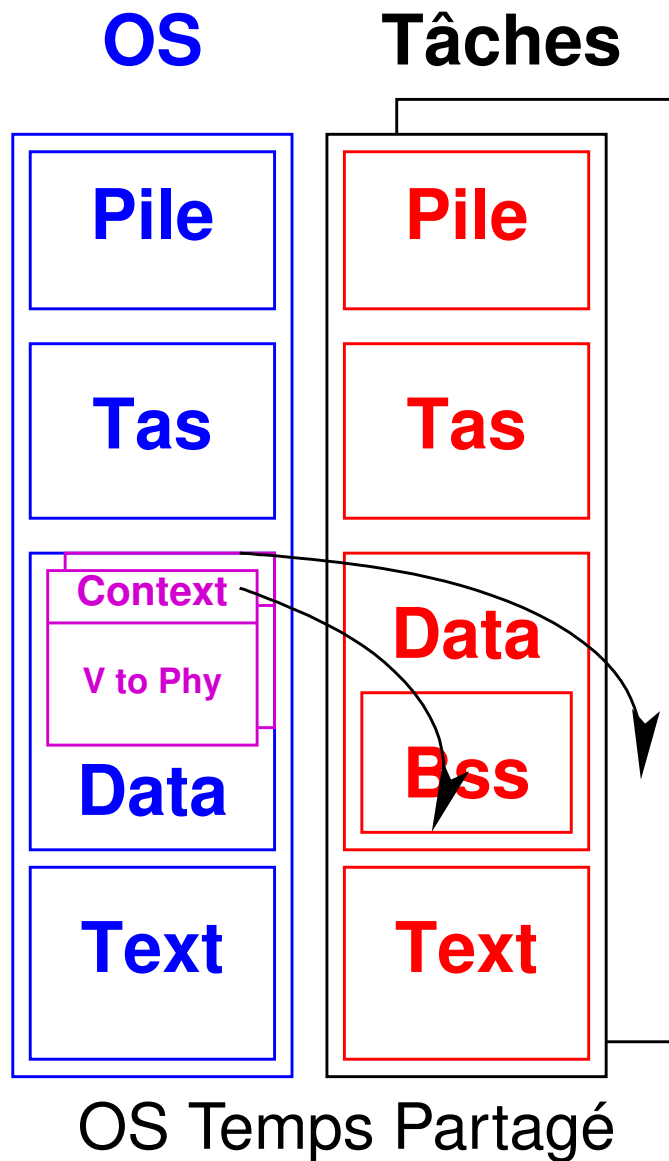
Etats des processus sous OS Temps Réel

- ❑ Un **OS Temps partagé** permet la création/destruction dynamique de processus
- ❑ Un **OS Temps réel** a déjà ses tâches dans sa section de code

Chaîne de compilation



Comparaison temps partagé/temps réel



Fonctionnement de l'OS temps-réel

L'OS temps réel:

- ❑ Se présente comme une bibliothèque de **fonctions**
- ❑ Un **service** de l'OS est un appel de fonction de l'OS (créer une tâche, attendre, etc ...)
- ❑ L'OS est démarré par un appel spécifique
- ❑ En cours de fonctionnement, l'OS implémente la **préemption** et la **politique d'ordonnancement**:
 - . Lancer une tâche
 - . Arrêter une tâche
 - . Gérer l'état d'une tâche
 - . Suspendre une tâche
 - . Sélectionner une tâche éligible
 - . Re-lancer une tâche

FreeRTOS est multi-tâche

```
int main(void) {
    /*
     * Fonctions d'initialisation des différentes entrées/sorties
     * et des protocoles de communication
     */
    BOARD_init_all();
    PRINTF("Hello World CCTR SEOC Test\n\r");
    /* Création d'une tâche avec pile statique, priorité statique */
    xTaskCreate(task_LED_R, "Task_LED_R",
               configMINIMAL_STACK_SIZE + 100, NULL,
               task_LED_R_PRIORITY, NULL);
    xTaskCreate(task_LED, "Task_LED_GB",
               configMINIMAL_STACK_SIZE + 100, NULL,
               task_LED_GB_PRIORITY, NULL);

    vTaskStartScheduler();
    /* Enter an infinite loop, but should never arrive here */
    for (;;)
        ;
    return 0 ;
}

/*
 * Tâche périodique pour faire clignoter la LED R
 */

void task_LED_R(void *pvParameters) {
    const TickType_t xDelay= Ms(100);
    led_cpt=0;

    for (;;)
    {
        LED_RED_TOGGLE();

        vTaskDelay(xDelay);
    }
}

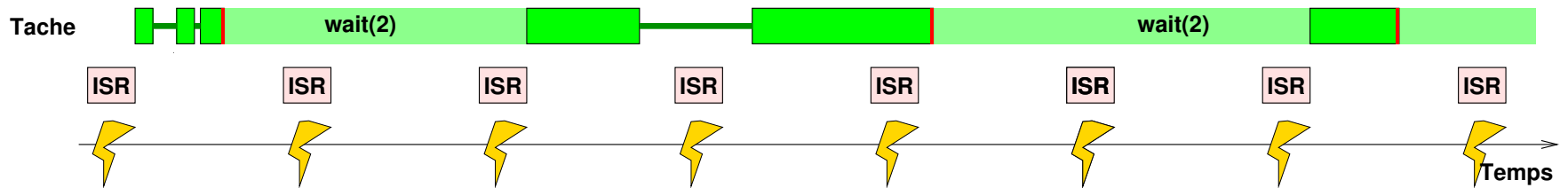
/*
 * Tâche périodique pour faire clignoter les LED GB
 */

void task_LED_GB(void *pvParameters) {
    const TickType_t xDelay= Ms(317);
    led_cpt=0;

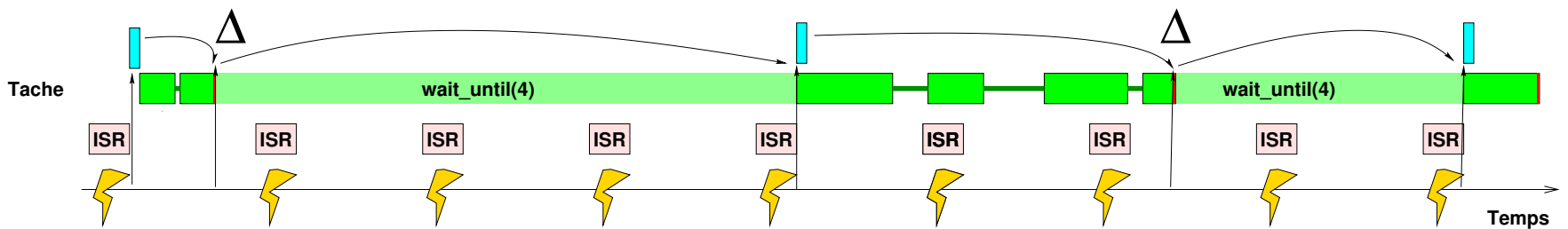
    for (;;)
    {
        LED_GREEN_TOGGLE();
        LED_BLUE_TOGGLE();

        vTaskDelay(xDelay);
    }
}
```

Stabilité des périodes



Période instable



Période stable

Implémentation des délais stables

Le mécanisme de délai insensible à la durée de calcul est le suivant:

```
const TickType_t xDelay= Ms(317);  
TickType_t xLastWakeTime;  
xLastWakeTime = xTaskGetTickCount();  
...  
for(;;) {  
    ... Calcul ...  
    vTaskDelayUntil(&xLastWakeTime, xDelay);  
}
```

➡ Sur le site de freeRTOS, consulter l'aide de la fonction **vTaskDelayUntil**

✖ Prise en main

- Rappels sur FreeRTOS

- ✓ C'est parti !!

□ Moteurs

□ Asservissement des moteurs

□ Contrôle de la direction

□ Planificateur de trajectoire

□ Gestion de la caméra

□ Suivi de la piste

Installation Polytech/OCAE

❑ Comptes

```
<usernames>: <xpe2i5a6{00,...,15}>  
<initial pw>; <Tre2i5a25>
```

❑ Ajouter un fichier `binome.txt` avec vos noms et prénoms

❑ Ouvrir un terminal texte

❑ Pour **installer** le projet, lancer les commandes

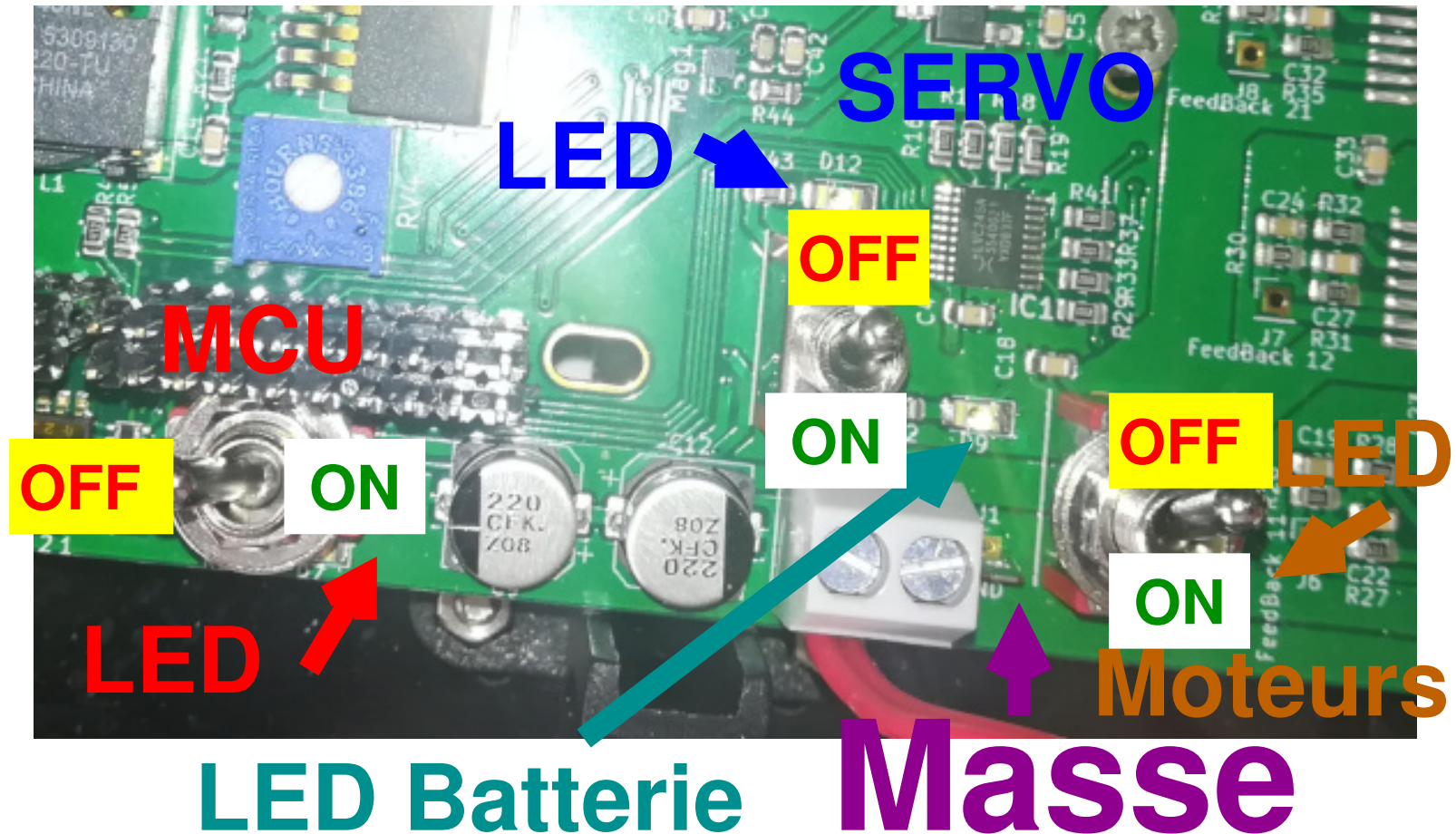
```
> cd ~  
> tar zxf /tp-fmr/smancini/CCTR_Polytech.tgz
```

❑ Pour **démarrer** l'environnement

```
> cd ~  
> source settings_mcuxpressoide-11.4.1_6260.sh
```

Puis sélectionner le 'workspace' `~/soft/CCTR/workspace`

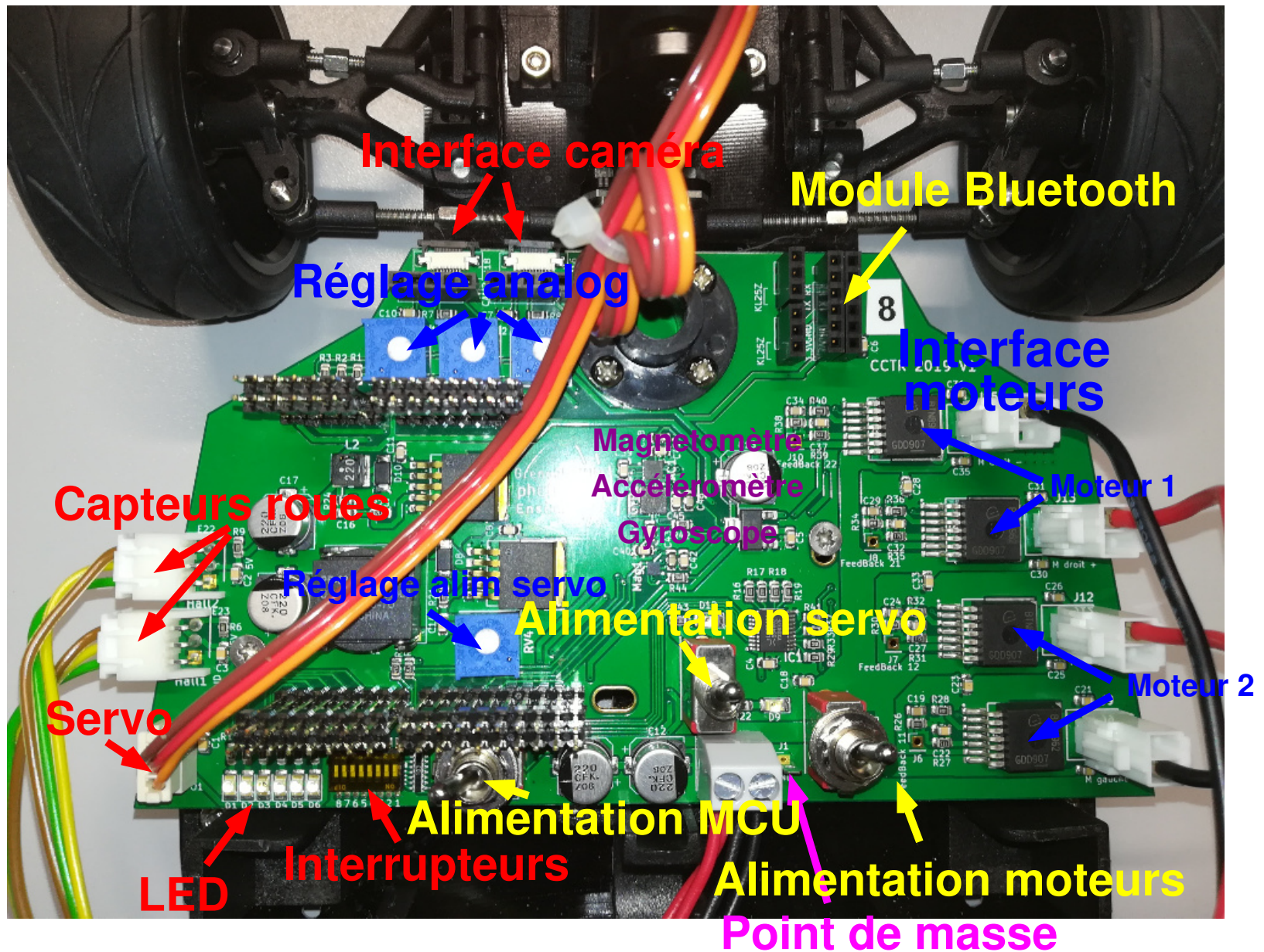
Alimentation



💣💣💣 Attention 💣💣💣

Eteindre tous les interrupteurs avant de
brancher la batterie

Carte d'interface MCU/Voiture

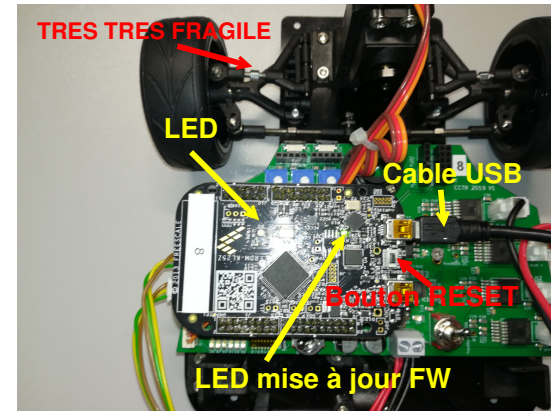


Let's go: se connecter au MCU

☞ Connectez la carte avec micro-USB sur le connecteur marqué **SDA**

☞ Dans un terminal, tapez

```
minicom -D /dev/ttyACM0  
ou  
minicom -D /dev/ttyACM1
```



☞ tapez : CTRL-A puis O

☞ A l'affichage du menu, sélectionner la configuration du modem

Serial port setup

☞ Lorsque les paramètres de configuration s'affichent tapez la lettre de chaque ligne et réglez pour obtenir:

Serial port setup

A -	Serial Device	:	/dev/ttyACM0	
B -	Lockfile Location	:	/var/lock	
C -	Callin Program	:		
D -	Callout Program	:		
E -	Bps/Par/Bits	:	115200 8N1	
F -	Hardware Flow Control	:	No	
G -	Software Flow Control	:	No	

Let's go: programmer le MCU

- 👉 Ouvrez MCUXpressoIDE par l'interface graphique
- 👉 Sélectionnez le "workspace" `soft/CCTR/workspace/`
- 👉 Clic droit projet `SEOC_CCTR` -> 'Clean Project'
- 👉 Dans la barre d'icônes, cliquez le marteau (build all)
- 👉 Le programme est compilé puis téléversé sur le microcontrôleur

✌️ Le programme est écrit en mémoire Flash, et le MCU démarre à la mise sous tension.

Des exemples d'accès aux périphériques et capteurs sont donnés. Ils seront détaillés au fur-et à mesure de l'évolution du projet.

Organisation des fichiers

- ❑ Fichier principal : `main_SEOC_CCTR.c`
Configuration du MCU, création des tâches et démarrage du RTOS
- ❑ Fichiers de tâches : `Task_<>.c`
- ❑ Fonctions de base et accès aux périphériques : `CCTR_<>.c`
Ne pas modifier les fichiers `CCTR_<>.c`
- ❑ Diverses bibliothèques

Lorsque de nouvelles tâches sont à créer, ajouter les fichiers qui vont bien dans le projet.

💣 Le code donné est un **exemple** que vous devez modifier!
💣 Créez de nouveaux fichiers et de nouveaux noms pour le projet.

Mission: programmer le MCU

Pour prendre en main l'environnement de programmation, nous allons programmer la LED RGB du MCU. Cette LED est composée de 3 LEDs, chacune commandée par un signal binaire.

N'allumer que l'interrupteur MCU, éteindre les autres.

Dans le fichier `main_CCTR_SEOC.c`

- ❑ Désactivez toutes les tâches en commentant les lignes de code adéquates
- ❑ N'activez que la tâche `Task_LED`
- ❑ Programmez la carte et observez la LED, ainsi que le texte dans le terminal
- ❑ Dans le code de la tâche LED, fichier `Task_LED.c`, modifiez le programme pour faire apparaître toutes les combinaisons de RGB, à une période de 0.25 seconde.
- ❑ Programmer la carte et vérifiez que le comportement est conforme

Observation de signaux internes

Il est utile de tracer l'évolution dans le temps des mesures, de l'état interne et des commandes.

❑ Sur la voiture

- . Dans les tâches, utilisez la fonction `PRINTF ("<key> %d\n\r", <val>)`

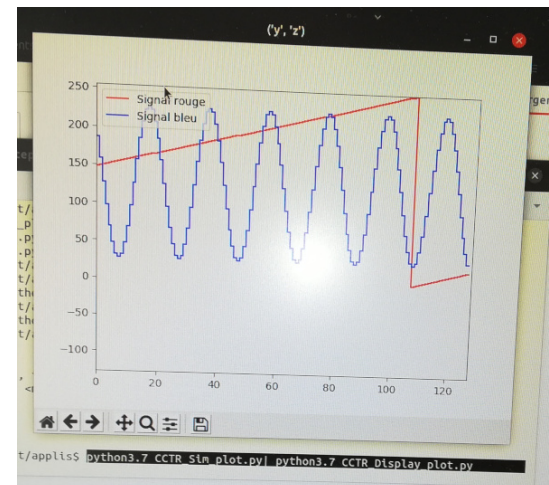
❑ Sur le PC

- . Dans le répertoire `~/soft/applis`, un exemple de code python pour afficher des suites de valeurs en provenance du MCU.
- . Affichage de signaux en provenance du port USB
`python3 CCTR_Display_plot.py /dev/ttyACM0`
- . Dans le code python les `<key>` sont déclarées au moment de créer la fenêtre d'affichage

Le code intercepte les chaînes de caractères et récupère les valeurs qui suivent une 'clé'.

Il est possible d'afficher simultanément plusieurs courbes, dans une même fenêtre ou dans plusieurs.

Lire le code et les commentaires pour plus d'informations.



Plan

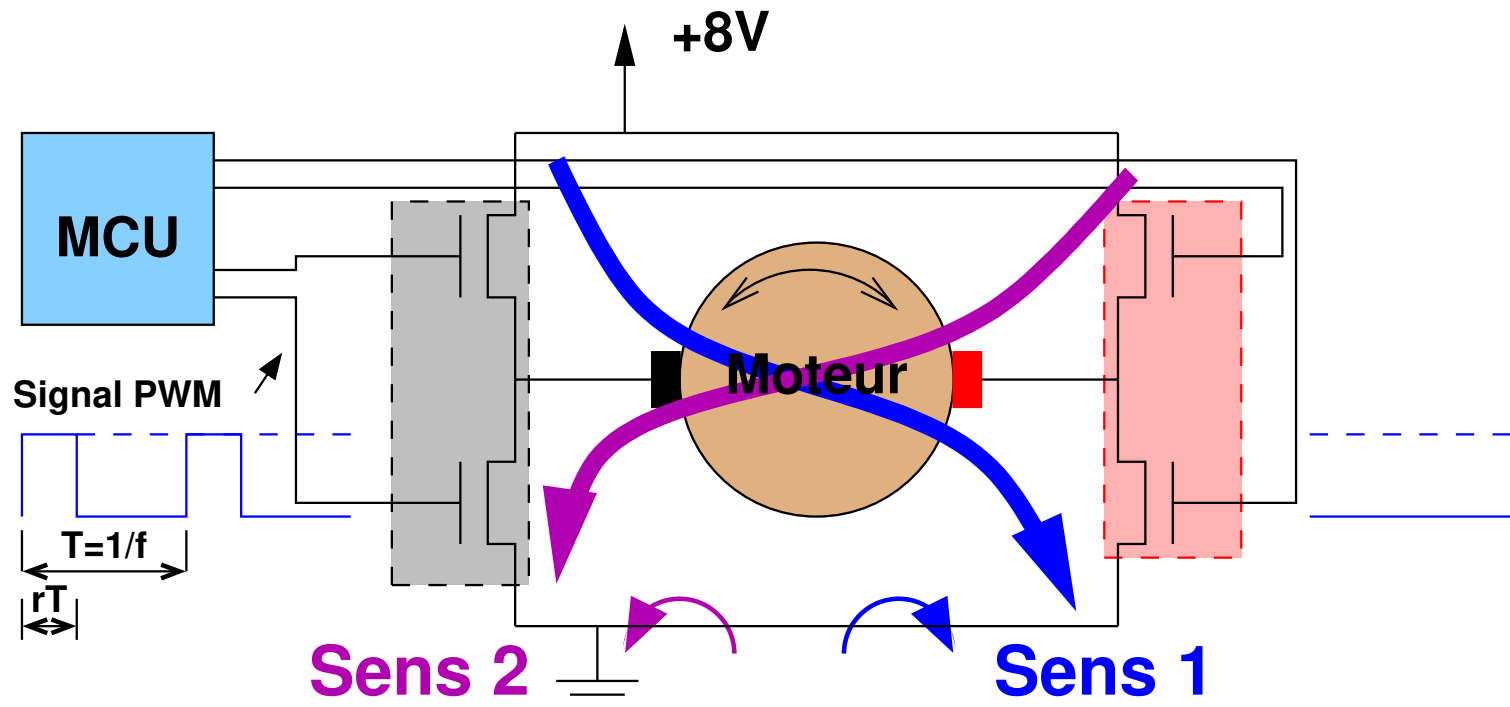
- ☐ Prise en main
- ✖ Moteurs
- ☐ Asservissement des moteurs
- ☐ Contrôle de la direction
- ☐ Planificateur de trajectoire
- ☐ Gestion de la caméra
- ☐ Suivi de la piste

Contrôle des moteurs

Les moteurs sont alimentés par un **pont en H**: des transistors FET de puissance envoient des courants forts dans les moteurs, commandés depuis le MCU.

Le pont en H est commandé par un signal binaire!

➡ Pour régler la puissance, on contrôle le rapport cyclique r d'un signal PWM.

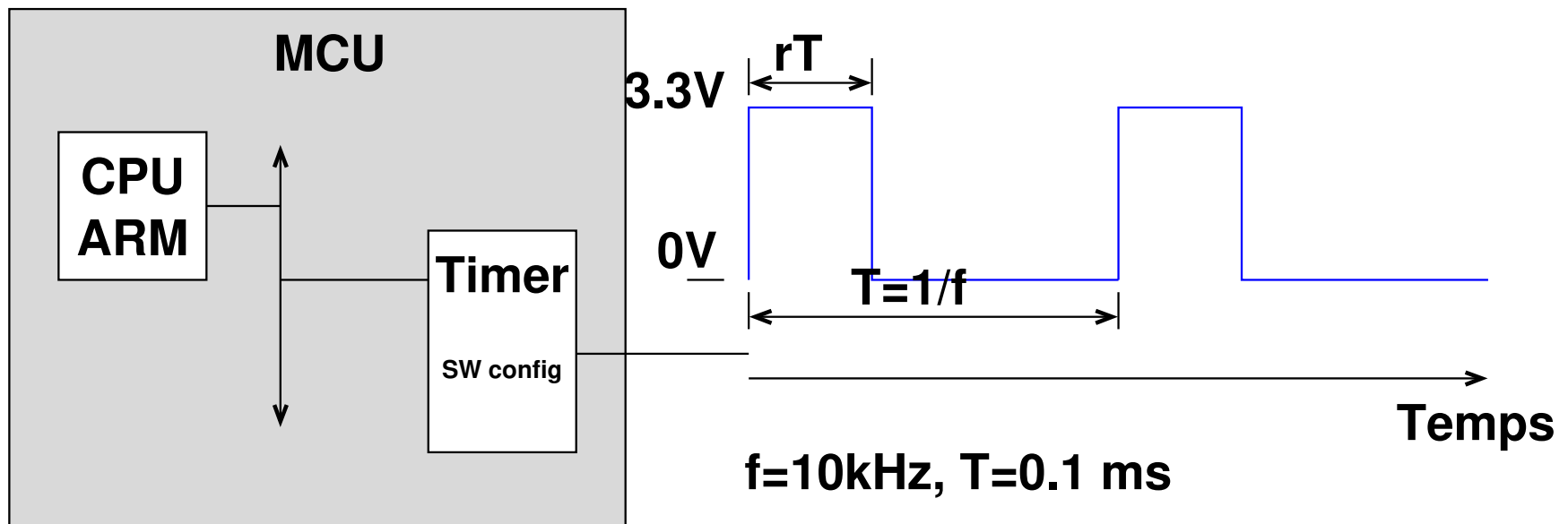


Génération du signal PWM

Le signal PWM est à 10kHz: il est trop rapide pour être généré en logiciel. Un périphérique 'Timer' génère le PWM et il est possible de régler la fréquence f et le rapport cyclique.

Une API logicielle permet de régler le rapport cyclique r .

Des exemples de codes sont dans les fichiers `CCTR_motor.c` et `Task_Motor.c`.



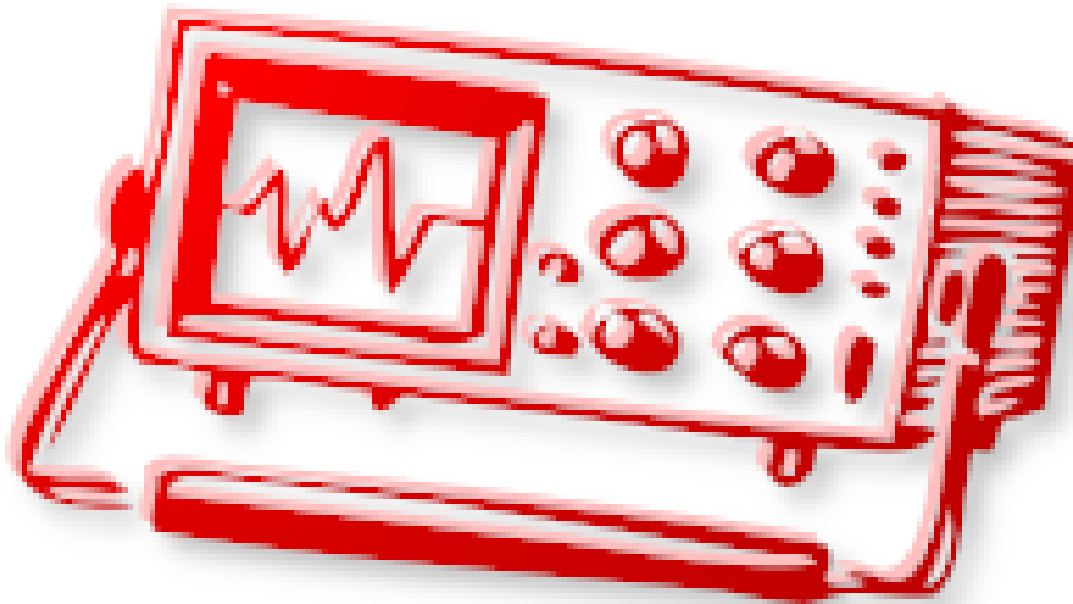
Mission : observer le signal PWM

- 👉 Hors tension,
 - Enlevez les connecteurs vers les moteur
 - Avec l'oscilloscope, , connectez la **masse de la sonde au point de masse** et mesurez le signal en sortie des interfaces de puissance
- 👉 Dans `main_SEOC_CCTR.c`, activez la tâche `Task_Motor`
- 👉 Programmer le MCU
- 👉 Allumez les alimentations MCU et Moteur
- 👉 Observez les signaux sur les 4 connecteurs vers les moteurs

   **Attention à la masse de l'oscilloscope**   

Mission : observer les roues

- 👉 Déconnectez l'oscilloscope et connectez à nouveau les moteurs
- 👉 Vérifiez que les roues se comportent comme indiqué dans le code



Mission : aller en ligne droite

- 👉 Dans le code de la tâche `Task_Motor`, réglez les moteurs pour avancer en ligne droite
- 👉 Avec des délais de Free-RTOS, faire en sorte de démarrer et s'arrêter au bout de 1m
- 👉 Puis reculer pour revenir au point de départ
- 👉 Idem sur 4m

Plan

- ☐ Prise en main
- ☐ Moteurs
- ✗ Asservissement des moteurs
- ☐ Contrôle de la direction
- ☐ Planificateur de trajectoire
- ☐ Gestion de la caméra
- ☐ Suivi de la piste

Constat

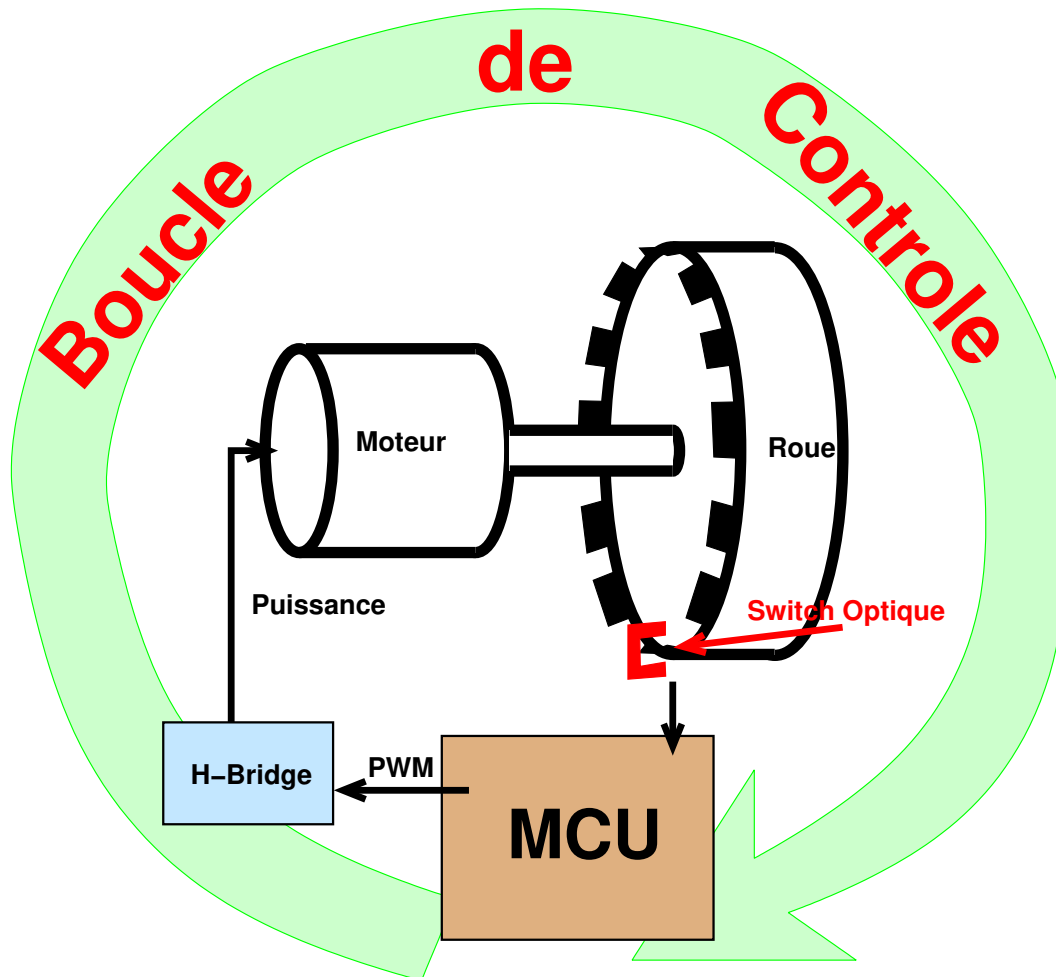
Il est impossible d'aller en ligne droite de cette façon !

- ❑ La voiture ne va pas droit car les roues ne sont pas exactement à la même vitesse
- ❑ Le système est très difficile à paramétrer, les valeurs sont différentes selon chaque voiture

- Il est nécessaire de mettre en place un **asservissement** de la voiture pour contrôler sa trajectoire!
- La voiture est contrôlée à partir de capteurs: vitesse des roues, accéléromètre, gyroscope et caméra ...

Contrôle des moteurs: principe

Afin de régler précisément la vitesse de rotation des moteurs, une boucle de contrôle ajuste le rapport cyclique r de façon à atteindre la vitesse voulue, en mesurant la vitesse des roues!



Mesure de la vitesse

La mesure de la vitesse est faite à l'aide d'un compte tour, réalisé à l'aide d'un capteur optique et d'ailettes placés sur la roue.

Le capteur détecte le passage des ailettes et on compte le nombre de passages pendant un certain intervalle de temps T_e , calculé selon la vitesse typique.

La vitesse angulaire s'exprime par $\omega = n \frac{2\pi}{pT_e}$ avec p le nombre d'ailettes sur la roue, n le nombre de passages pendant l'intervalle T_e .

La vitesse de la voiture est $v = r\omega$, avec r le rayon de la roue.

Implémentation logicielle de la mesure de vitesse

Il serait trop complexe de vérifier en permanence l'état des capteurs: la tâche de mesure consommerait tout le temps processeur.

➡ Les capteurs sont connectés à des interruptions.

A chaque passage devant le capteur, le code associé à l'interruption (ISR) est exécuté.

L'ISR

Incrémente un compteur associé à chaque capteur

La tâche de mesure

- Récupère périodiquement la valeur de ce compteur: le nombre de passage pendant la période
- Remet le compteur à zéro pour la prochaine période

Bruit et précision de la mesure de vitesse

La mesure de vitesse est conditionnée par:

- ❑ Le nombre d'ailettes => la résolution angulaire

Pour l'améliorer, il est possible de faire la détection en amont du réducteur

- ❑ La période de mesure => adaptée à la vitesse

Plus le moteur est rapide, plus la mesure est précise mais les IT ralentissent le processeur!

- ❑ La stabilité de la période de la tâche de mesure => effet sur la vitesse mesurée

Les variations de période sont interprétées comme des variations de vitesse

- ❑ La synchronisation ISR/Tâches => perte de quelques passages

La tâche de mesure peut être interrompue entre la lecture du compteur et sa remise à zéro; de rares détections peuvent être perdues.

- ❑ La vitesse de rotation et la période de la tâche ne sont pas en phase

Même à vitesse constante, le nombre de détections change entre deux périodes de mesure.

- La période de mesure T_e est liée à la vitesse ciblée.
- Pour ne pas générer des a-coups, l'asservissement ne doit pas transformer le bruit de mesure en bruit sur la commande.

Boucle d'asservissement

En théorie du contrôle-commande une boucle d'asservissement permet de forcer l'état d'un système physique dynamique en modulant les entrées de ce système à partir de mesures des sorties.

Le contrôle optimal est une théorie mathématique complexe et nécessite la connaissance de paramètres du système.

Pour faire simple, on met en place une commande 'intégrale' de la forme:

$$\text{PWM}_{k+1} = \text{PWM}_k - K_s(\omega - \omega_{\text{ref}})$$

Avec

- ❑ ω , la vitesse angulaire mesurée
- ❑ ω_{ref} , la vitesse angulaire cible
- ❑ K_s , le gain de la boucle de contrôle
- ❑ PWM, le rapport cyclique de la commande du moteur

Boucle ouverte, sans asservissement

Dans le fichier `Task_Speed.c`,
le canevas de tâche `task_Speed` illustre la mesure de la vitesse.

Le nombre de détection mis à jour par l'ISR est accessible par les variables `speed_<1 | 2>`

- 👉 Dans la tâche `task_Motor`, réglez le rapport cyclique pour que le moteur tourne 'raisonnablement' (ie 20%).
- 👉 Pour observer la vitesse, activez la tâche `task_Speed` et lancez la commande

```
python3.4 applis/CCTR_Display_motor.py /dev/ttyACM0
```

- 👉 Programmez le MCU et, ensuite, freinez la roue avec la main et observez l'évolution de la vitesse

Boucle d'asservissement

Dans la tâche `task_Speed`:

- 👉 Mettez en place la boucle d'asservissement
- 👉 Essayez différentes valeurs de K_s
par ex. 10^{-5} , 10^{-4} , 10^{-3} , 10^{-2} , 10^{-1} , 1.0 , ...
- 👉 Visualisez la vitesse et la valeur de PWM, en freinant la roue
`PRINTF ("MG %d\n\rMD %d\n\r", pwm_1, pwm_2);`

👉 Selon la valeur de K_s , le moteur réagit très lentement ou oscille fortement.

👉 La tâche `task_Motor` doit être désactivée pour ne pas perturber l'asservissement

Contrôle de la voiture

-  Mettez en place deux boucles de contrôle, une pour chaque moteur, et essayez de faire avancer la voiture en ligne droite.
-  Programmez la voiture pour qu'elle avance d'un mètre puis recule d'un mètre
-  C'est facile maintenant que vous connaissez sa vitesse et la distance parcourue!!
-  Calculez le temps pendant lequel la voiture avance
-  ... ou bien ...
-  Contrôlez la vitesse selon la distance parcourue

 Avec les techniques de contrôle avancées on peut aussi
 faire en sorte que les deux roues soient exactement à la
 même vitesse en permanence.

Plan

- ☐ Prise en main
- ☐ Moteurs
- ☐ Asservissement des moteurs
- ✗ Contrôle de la direction
- ☐ Planificateur de trajectoire
- ☐ Gestion de la caméra
- ☐ Suivi de la piste

Contrôle de la direction

La direction est réalisée par un servo-moteur de modélisme: la commande est un signal PWM de période 20 ms (50 Hz) et de durée entre 1 ms et 2 ms, 1,5 ms étant la position au repos.

Une API est fournie pour gérer la direction, dans le fichier `CCTR_steering.<c|h>`.

L'angle des roues est associé à une valeur de rapport cyclique au 10 000 ème comprise entre 500 et 1000.

⚡💣 Entre 750 et 900 pour éviter de détruire la direction. ☢️💀

La direction est gérée par les fonctions:

- ❑ `Steering_UpdatePwm_Relative`
 - . Direction **relative** à la position droite calibrée
 - . Retourne la direction effective, après clamping entre `STEERING_OFFSET-STEERING_AMPLITUDE` et `STEERING_OFFSET+STEERING_AMPLITUDE`
- ❑ `Steering_UpdatePwm_Absolute`
 - . Direction **absolue** du servo
 - . Retourne la direction effective, après clamping entre `STEERING_MIN` et `STEERING_MAX`

Installation de la direction

L'arbre de sortie du servomoteur est cranté! Cela rend impossible de régler une direction droite lorsque le servo est à sa position centrale!!

 Il est nécessaire de **calibrer** la direction pour obtenir une direction droite de référence.


-  Positionnement mécanique approché de la position centrale du servo
-  Réglage fin par logiciel pour calibrer la direction droite

Calibrage de la direction: logiciel

💣⚡ Attention, à faire **soigneusement** 💣⚡

Pour régler le 'zéro' logiciel de la direction, suivez les instructions dans la tâche de `Task_Steering.c`

- 👉 **Lire tous les commentaires** du code de `Task_Steering.c` et positionnez les potentiomètres P2 et P3 pour activer le réglage de la ligne droite
- 👉 Faites varier le potentiomètre de gauche (P1) à l'aide du tournevis
- 👉 Notez la valeur affichée sur le terminal lorsque les roues sont droites
- 👉 Reportez cette valeur dans le code du fichier

`CCTR_steering.h` **En modifiant la macro** `STEERING_RELATIVE_OFFSET`

- 💣 **Réglez l'amplitude maximum** pour que les tiges de direction **ne touchent pas le châssis**

Démonstration de la direction

Après avoir calibré la direction, modifié `CCTR_steering.h` et recompilé le code, vous pouvez tester la démonstration:

- 👉 Dans la tâche `Task_Steering`, avec les potentiomètres, activez la portion de code de démo
- 👉 Vérifiez que la direction ne touche pas le châssis, et, le cas échéant, recommencez la calibration !

👉 Pour un calibrage avancé, on peut aussi tracer la courbe de l'orientation des roues en fonction du rapport cyclique.

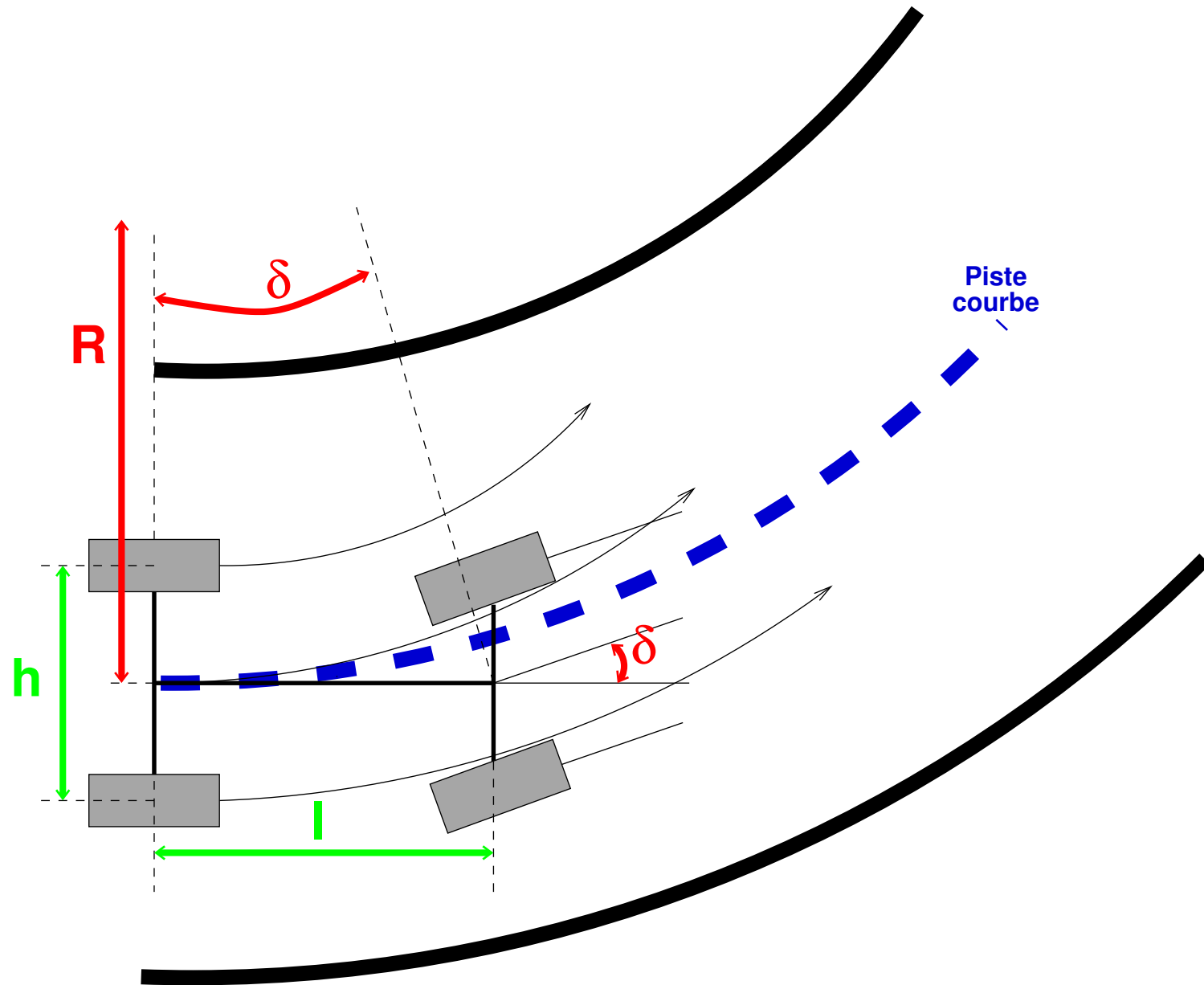
Différentiel logiciel

Dans un virage, les roues avant et arrière ne vont pas à la même vitesse:

- ❑ Les roues internes vont moins vite
- ❑ Les roues externes vont plus vite

- ? Etablissez la relation entre les vitesses des roues internes et externes
- ?
 - ❑ La voiture évolue sur une courbe de rayon R
 - ❑ La longueur de la voiture est l , sa largeur h
- ?
 - ❑ Les roues avant sont orientées de l'angle δ
 - ❑ Les roues ont un rayon r

Différentiel Géométrique



Contrôle des moteurs

La tâche de contrôle de moteurs implémente le différentiel en permettant des consignes (au choix):

- En absolu

Chaque roue reçoit une consigne séparée.

- En relatif

Une roue sert de référence, la seconde consigne est relative à cette référence.

- En vitesse moyenne et courbure

La tâche calcule les références absolues/relatives de chaque roue.

➤ Utilisez la direction et le différentiel logiciel pour prendre un virage, en **préservant la vitesse moyenne!**

Implémentation du différentiel

La tâche de gestion des roues reçoit des consignes depuis les autres tâches (planification, etc ...).

Pour faire simple, la consigne est donnée sous forme de variable globale, modifiée par les autres tâches.

Modèle avancé

Il est possible de réaliser le contrôle simultané des roues, en prenant en compte l'inertie de la voiture.

⇒ Contrôle multi-variable par contrôleur de Kalman optimal

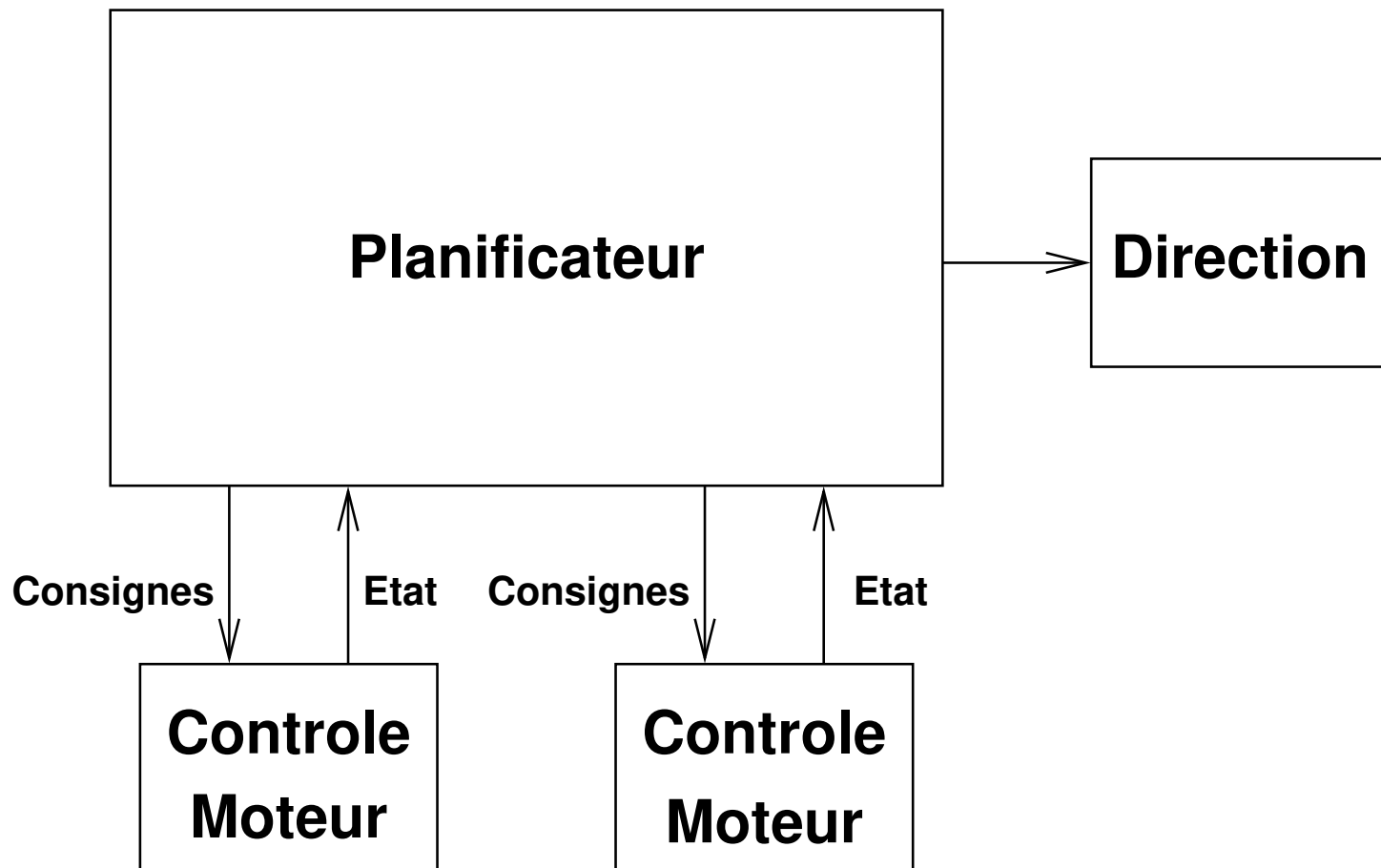
Mais cela est difficile à implémenter car nécessite une connaissance précises de tous les paramètres du modèle dynamique (voir poly associé).

Plan

- ☐ Prise en main
- ☐ Moteurs
- ☐ Asservissement des moteurs
- ☐ Contrôle de la direction
- ✗ Planificateur de trajectoire
- ☐ Gestion de la caméra
- ☐ Suivi de la piste

Planificateur de trajectoire

Un planificateur de trajectoire est une tâche qui fait évoluer les consignes de direction et de vitesse pour suivre une trajectoire prédéfinie.

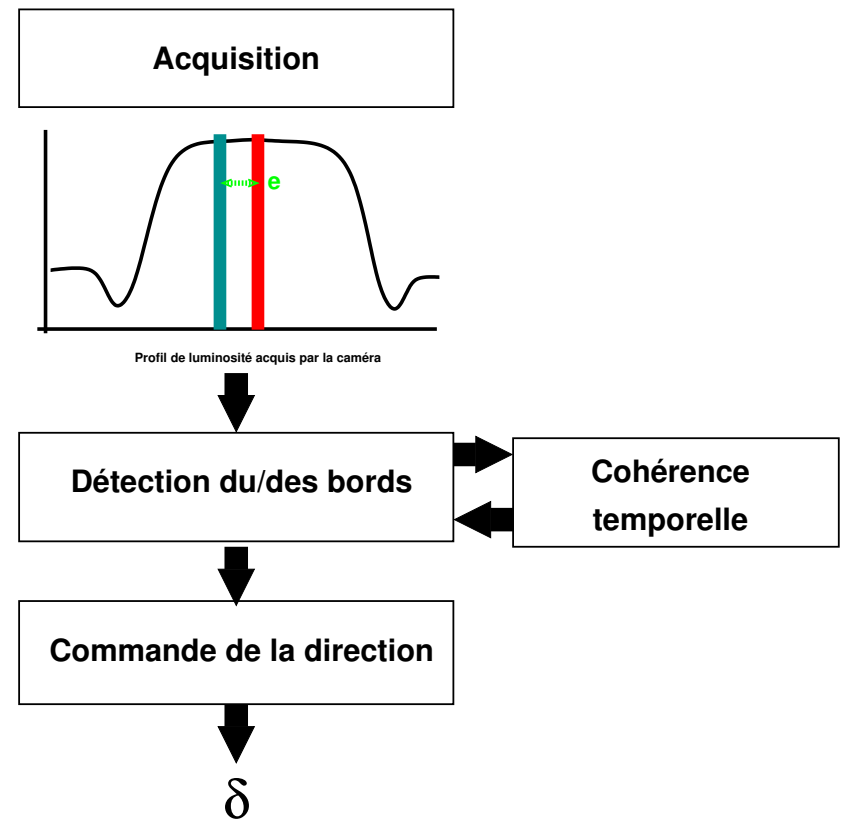
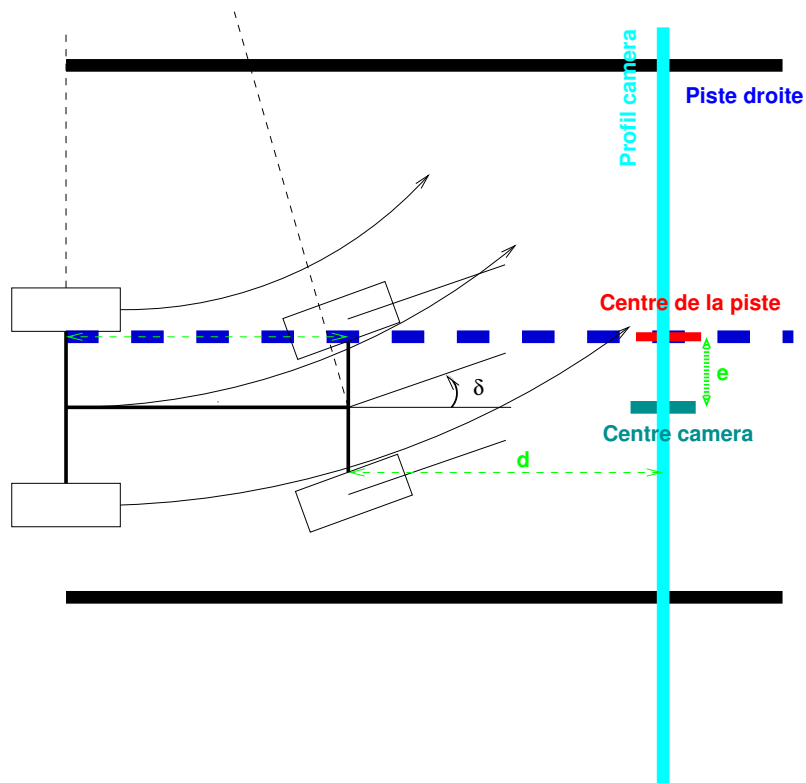


Plan

- ☐ Prise en main
- ☐ Moteurs
- ☐ Asservissement des moteurs
- ☐ Contrôle de la direction
- ☐ Planificateur de trajectoire
- ✗ Gestion de la caméra
- ☐ Suivi de la piste

Suivi de la piste

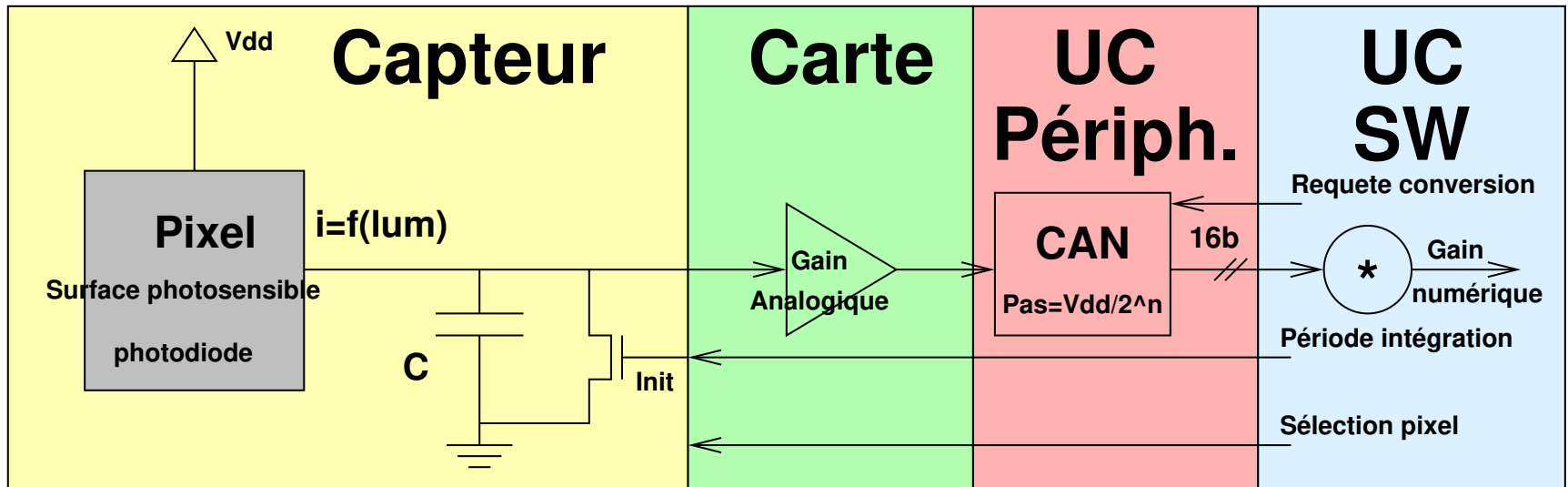
Suivre la piste consiste à détecter ses bords sur l'image de la caméra et contrôler la direction de la voiture pour la centrer sur la piste.



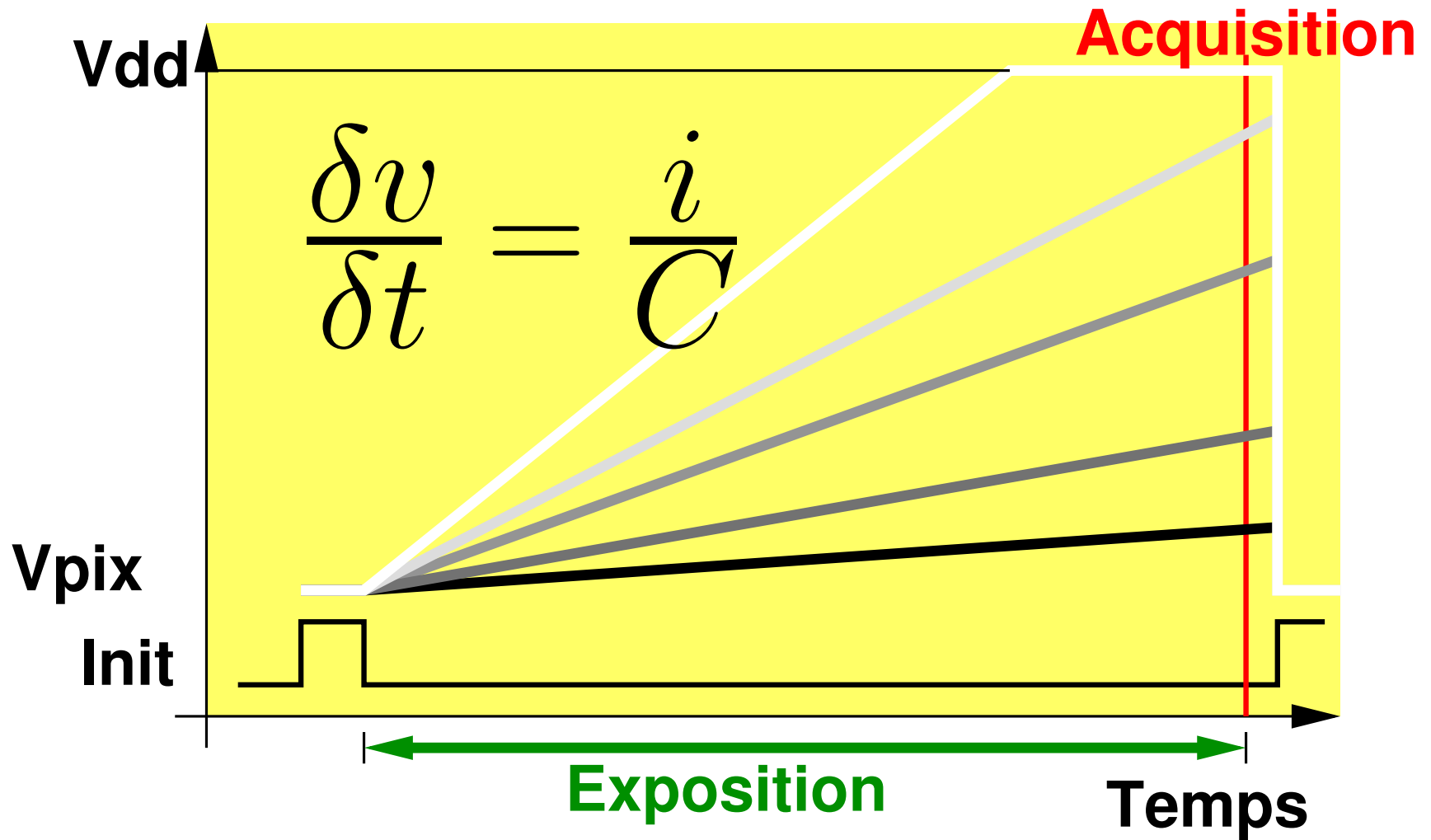
La caméra

La caméra de la voiture est une caméra ligne **analogique**. Elle permet l'acquisition d'une tranche de 128 pixels devant la voiture.

La conversion analogique/numérique est réalisée dans le micro-contrôleur, par un **CAN** (ADC).



Transduction photon-électron



Gestion du temps d'exposition

Pour acquérir une ligne il faut:

- ❑ Déclencher l'intégration (init)
- ❑ Attendre la charge des pixels par la conversion photon/électron de la photodiode
- ❑ Un gain analogique peut améliorer le niveau de signal/bruit
- ❑ Récupérer la valeur de chaque pixel et en faire la conversion analogique/numérique

Dans le micro-contrôleur, le CNA convertit une tension en un pas de la tension d'alimentation. La résolution peut être choisie (8, 10 ou 12 bits).

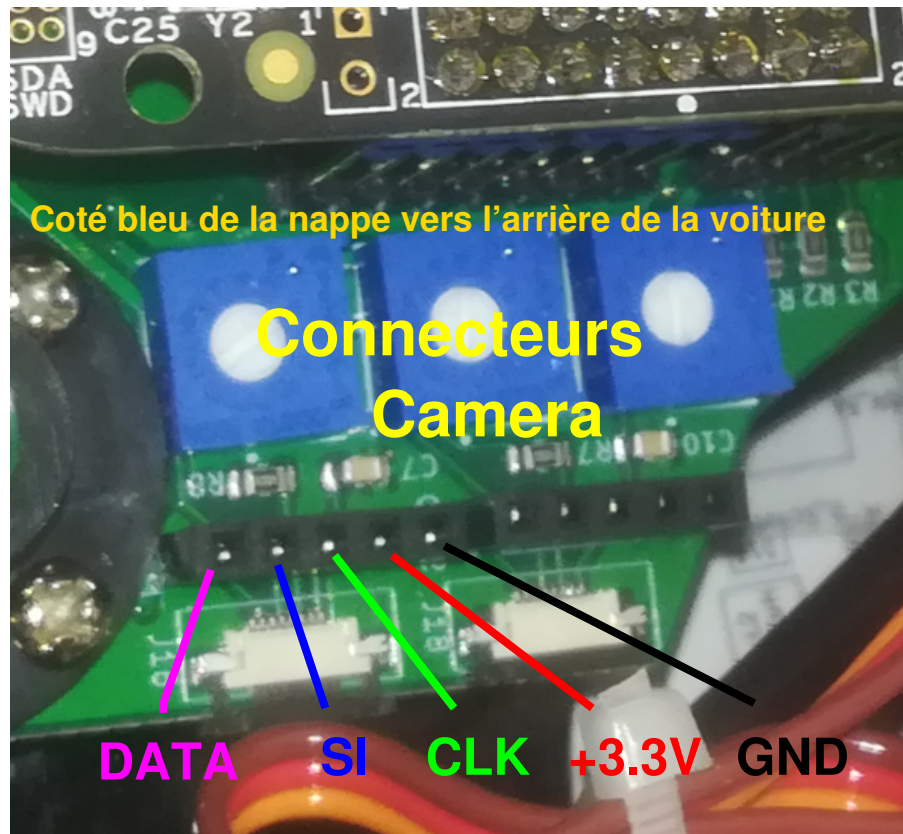
- ➤ Le réglage des gains et temps d'exposition dépend des conditions lumineuse
- ➤ La période d'intégration doit être stable car elle agit comme un gain

Prise en main de la caméra

- ➡ Analysez le code de la tâche `Task_Camera`
- ➡ Activez les tâches `Task_Camera` et `Task_Camera_Display`
- ➡ Sur le PC, visualiser la ligne acquise:

```
python3 CCTR_Display_camera.py /dev/ttyACM0
```

Lors de l'affichage de la caméra il est possible d'ajouter des curseurs verticaux.



Synchronisation de la caméra

Le temps d'exposition en conditions lumineuses "normales" est d'environ 8 ms.

➡ La tâche de gestion de la caméra occupe tout le temps processeur!

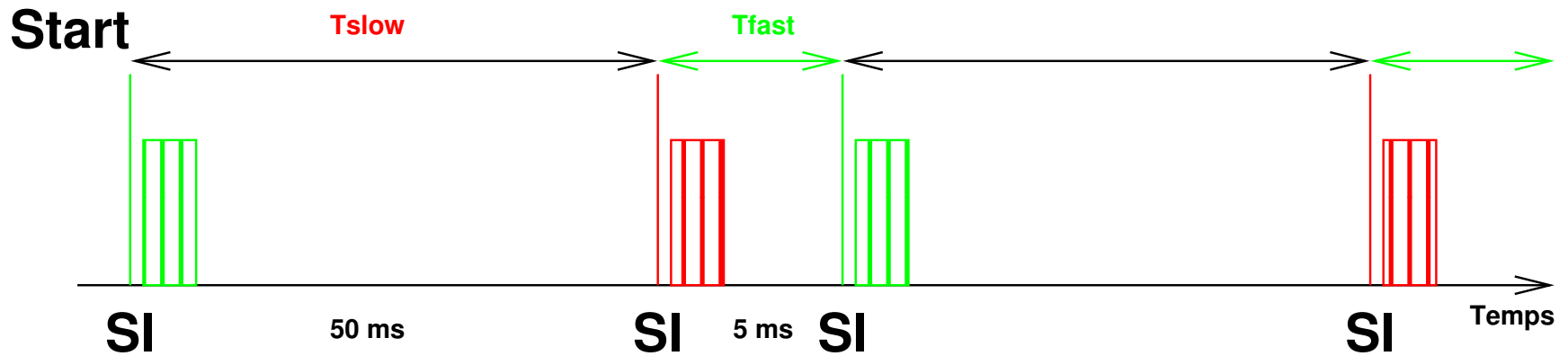
Il est nécessaire de mettre en place une double intégration:

- ❑ Une première de période 30 à 50 ms, dont le résultat n'est pas pris en compte
- ❑ Une seconde de 5 à 6 ms, dont les valeurs sont conservées

👍 Une stratégie élaborée consiste à adapter le temps d'intégration aux conditions lumineuses.

Double intégration

👉 Mettez en place la double intégration



Plan

- ☐ Prise en main
- ☐ Moteurs
- ☐ Asservissement des moteurs
- ☐ Contrôle de la direction
- ☐ Planificateur de trajectoire
- ☐ Gestion de la caméra
- ✗ Suivi de la piste

Suivi de la piste

Pour suivre la piste, l'algorithme:

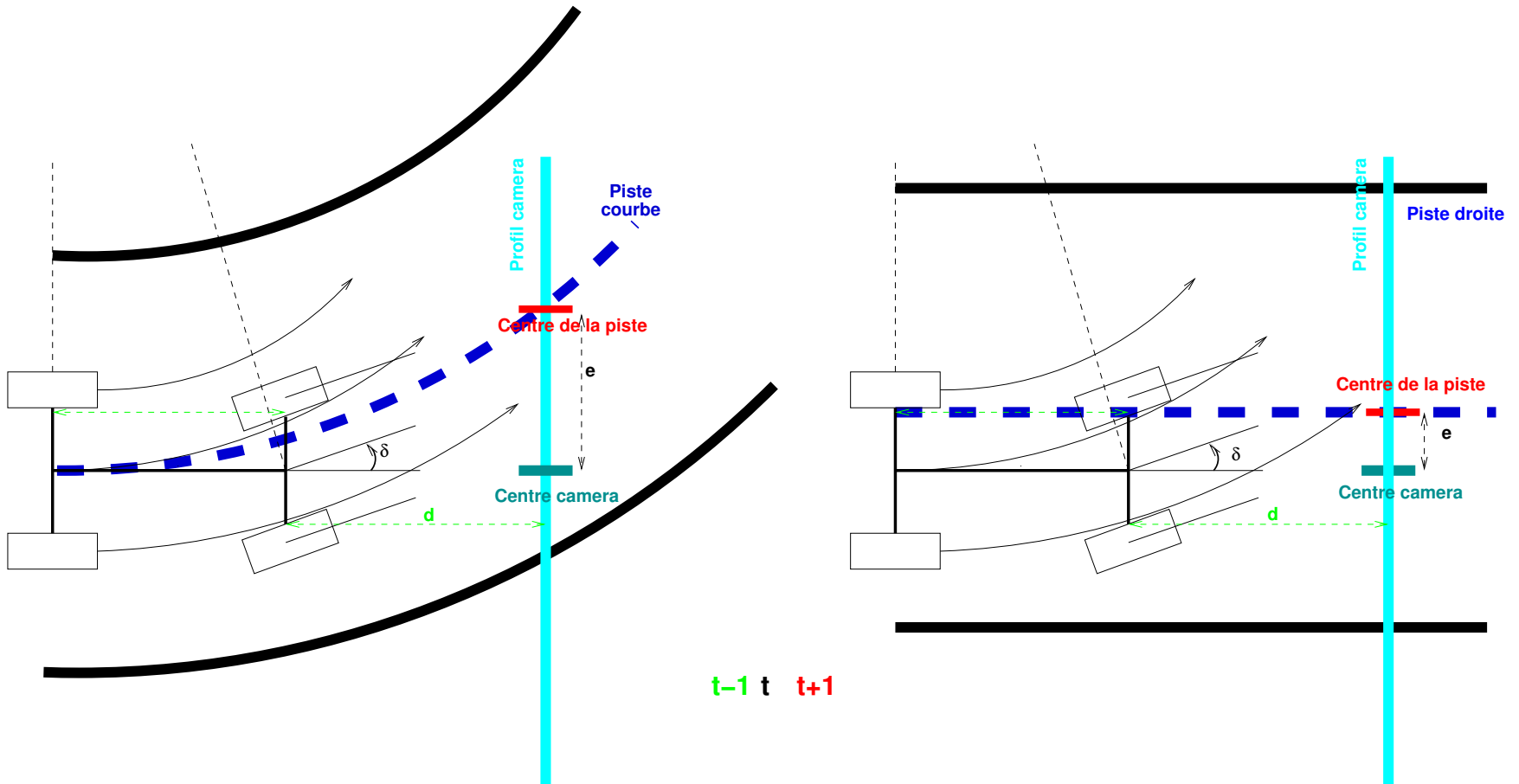
- ❑ Détecte les bords de la piste sur la caméra
- ❑ Règle la direction en conséquence

? Quelle stratégie adopter pour:

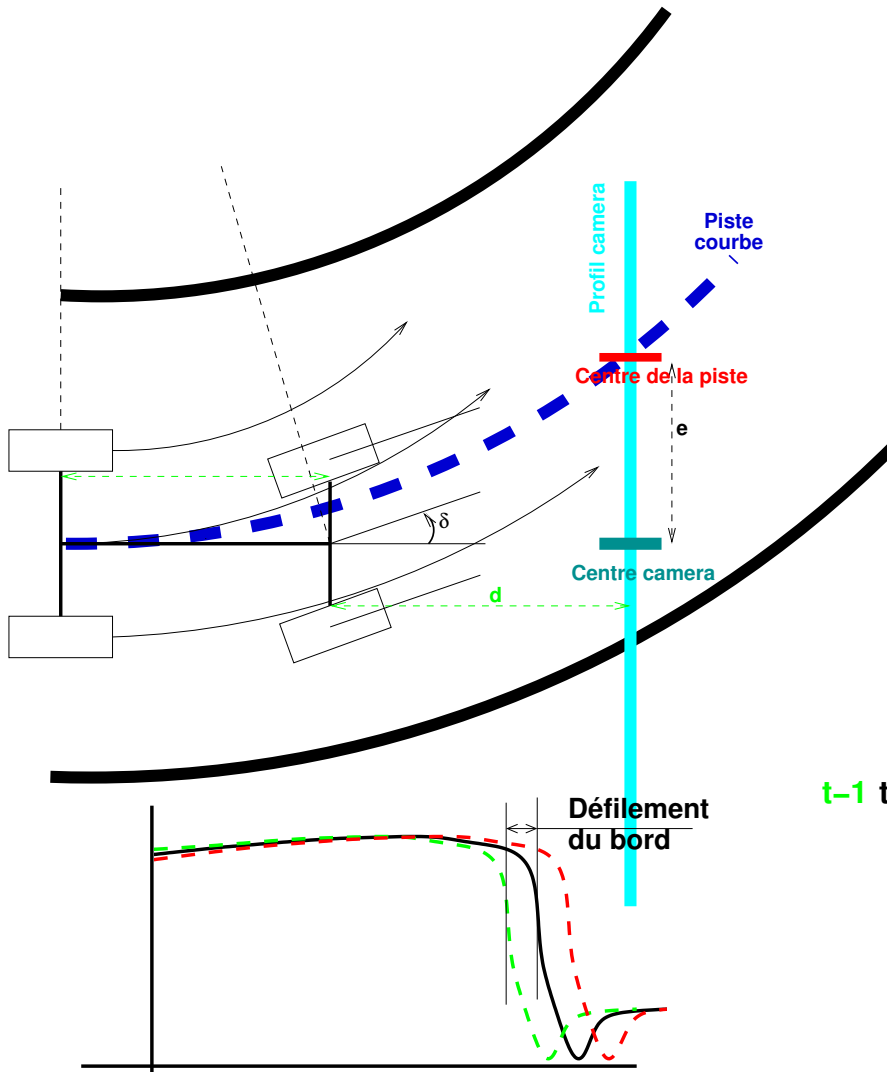
- ❑ Se centrer sur la piste en ligne droite
- ? ❑ Prendre un virage

avec une caméra qui regarde devant la voiture ?

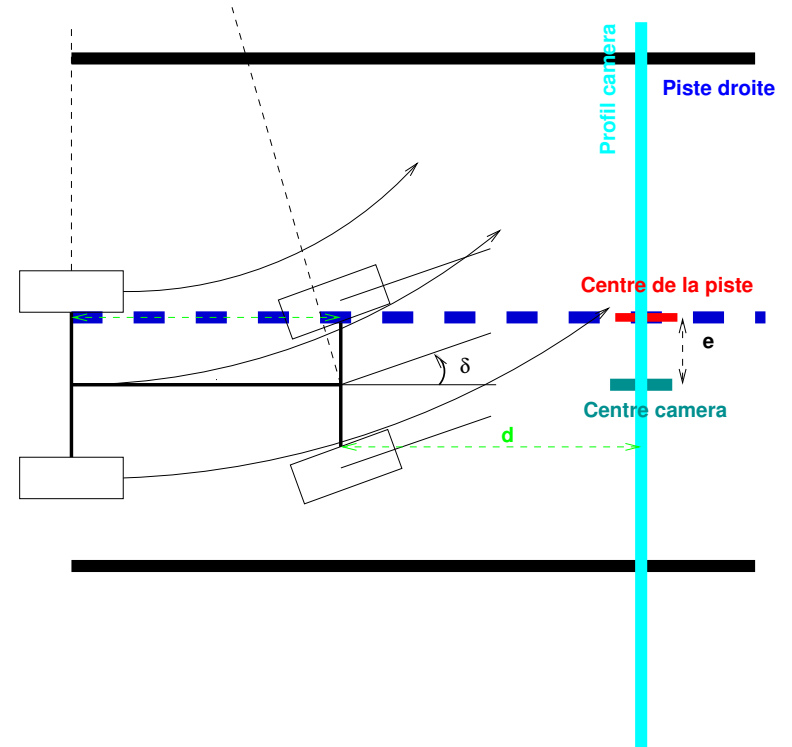
Modèle géométrique de la caméra



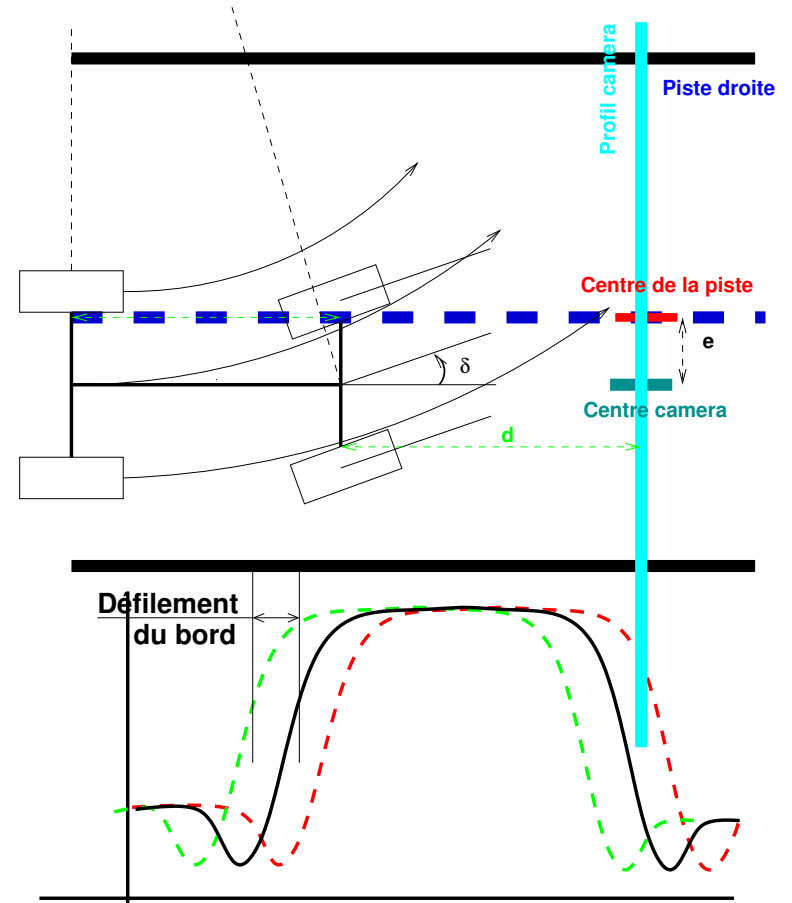
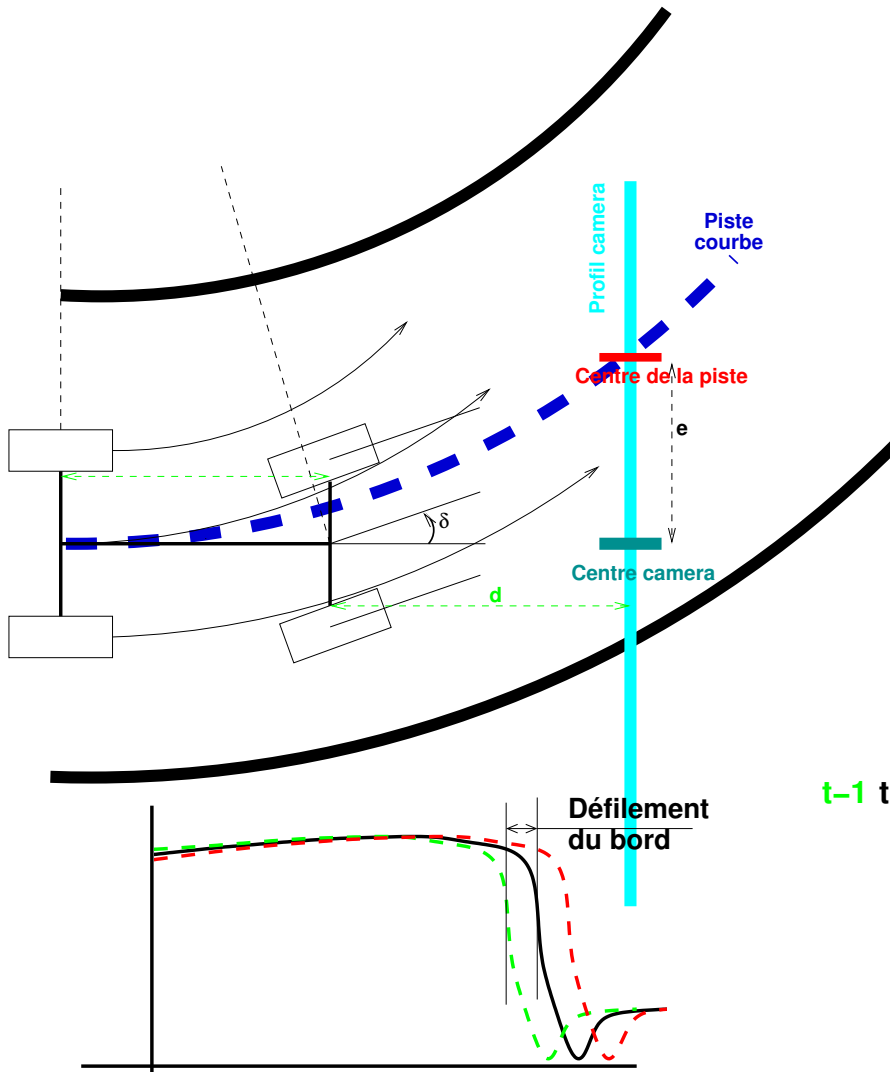
Modèle géométrique de la caméra



$t-1$ t $t+1$



Modèle géométrique de la caméra



Contrôle de la position sur la piste

La caméra peut détecter soit

- ❑ 2 bords,
- ❑ 1 bord gauche ou un bord droite
- ❑ ou aucun bord (on est perdu ??)
- ❑ En ligne droite,
 - . la voiture est centrée lorsque le milieu des bords est au centre de la caméra
 - . la voiture est parallèle à la piste lorsque les bords ont un défilement nul
- ❑ Dans un virage,
 - . la voiture suit la piste lorsqu'un des bords a un défilement nul

Suivi de la piste

La loi de contrôle minimise une erreur de suivi:

$$\delta_{k+1} = \delta_k - K_\delta \Delta$$

- ❑ δ_k , direction de la voiture à l'itération k
Unité: rapport cyclique PWM
- ❑ Δ est le défilement du milieu des bords ou d'un des bords
Unité: # pixels par période
- ❑ K_δ est le gain de la boucle de contrôle

Suivi de la piste

A l'aide de l'API du fichier `CCTR_camera.c`, et des fonctions de détection des bords,

- 👉 Réglez la vitesse de la voiture 0,25 m/s dans la tâche `Task_Motor`
- 👉 Mettez en oeuvre une première loi de commande de la direction basée sur la minimisation de la vitesse des bords dans la tâche `Task_Steering`
- 👉 Testez la voiture sur la piste

On constate une dérive progressive de la voiture et il est nécessaire de la centrer sur la piste lorsqu'elle est en ligne droite (c'est impossible en courbe).

Une solution consiste à combiner les deux critères:

- ❑ Annuler le défilement des bords Δ
- ❑ Annuler l'erreur de position latérale ϵ

Suivi amélioré

La loi de contrôle minimise une erreur de centrage et stabilise la voiture:

$$\delta_{k+1} = \delta_k - K_\delta c$$

$$c = \alpha \Delta + \beta \epsilon$$

- ❑ Δ le défilement d'un des deux bord
- ❑ ϵ l'erreur de position latérale
- ❑ avec α proche de 1 et β assez petit ...

Contrôle-commande temps réel des systèmes Cyber-Physiques

S. Mancini

Plan Détaillé

- ☆ Résumé du module
- ☆ Objectifs du projet
- ☆ Moyens pour le projet
- ☆ Missions
- ☆ Micro-contrôleur
- ☆ Logiciel multi-tâche!

✖ Prise en main

- Rappels sur FreeRTOS
 - ☆ Application sous FreeRTOS
 - ☆ Gestion des processus
 - ☆ Chaîne de compilation
 - ☆ Comparaison temps partagé/temps réel
 - ☆ Fonctionnement de l'OS temps-réel
 - ☆ FreeRTOS est multi-tâche
 - ☆ Stabilité des périodes
 - ☆ Implémentation des délais stables
- C'est parti !!
 - ☆ Installation Polytech/OCAE
 - ☆ Alimentation
 - ☆ Carte d'interface MCU/Voiture
 - ☆ Let's go: se connecter au MCU

- ☆ Let's go: programmer le MCU
- ☆ Organisation des fichiers
- ☆ Mission: programmer le MCU
- ☆ Observation de signaux internes

✖ Moteurs

- ☆ Contrôle des moteurs
- ☆ Génération du signal PWM
- ☆ Mission : observer le signal PWM
- ☆ Mission : observer les roues
- ☆ Mission : aller en ligne droite

✖ Asservissement des moteurs

- ☆ Constat
- ☆ Contrôle des moteurs: principe
- ☆ Mesure de la vitesse
- ☆ Implémentation logicielle de la mesure de vitesse
- ☆ Bruit et précision de la mesure de vitesse
- ☆ Boucle d'asservissement
- ☆ Boucle ouverte, sans asservissement
- ☆ Boucle d'asservissement
- ☆ Contrôle de la voiture

✖ Contrôle de la direction

- ☆ Contrôle de la direction
- ☆ API de la direction
- ☆ Installation de la direction
- ☆ Calibrage de la direction: logiciel
- ☆ Démonstration de la direction
- ☆ Différentiel logiciel
- ☆ Différentiel Géométrique
- ☆ Contrôle des moteurs
- ☆ Implémentation du différentiel
- ☆ Modèle avancé

✖ Planificateur de trajectoire

- ☆ Planificateur de trajectoire

✖ Gestion de la caméra

- ☆ Suivi de la piste
- ☆ La caméra
- ☆ Transduction photon-électron
- ☆ Gestion du temps d'exposition
- ☆ Prise en main de la caméra
- ☆ Synchronisation de la caméra
- ☆ Double intégration

✖ Suivi de la piste

- ☆ Suivi de la piste
- ☆ Modèle géométrique de la caméra
- ☆ Contrôle de la position sur la piste
- ☆ Suivi de la piste
- ☆ Suivi de la piste
- ☆ Suivi amélioré
- ☆ Suivi amélioré